

Hochschule für Technik und Wirtschaft Berlin

Studiengang Computer Engineering

Bachelorarbeit

**Untersuchung von Watchdog- und
Recovery-Strategien für ein robustes
eingebettetes System**

vorgelegt von

Omar Altanbakji

Matrikelnummer S0587682

Erstgutachter Prof. Dr. Jochen Kerdels

Zweitgutachter Prof. Dr. Marcin Brzozowski

Abgabedatum 21. September 2026

Zusammenfassung

Diese Bachelorarbeit untersucht Watchdog- und Recovery-Strategien an einem heterogenen eingebetteten System aus Raspberry Pi und dem Mikrocontroller STM32L432KC. Als Testanwendung erfasst der Raspberry Pi Positionsdaten eines Binghe VK-162 USB-GPS-Empfängers, ruft Wetterdaten für den ermittelten Standort ab und überträgt einen strukturierten Status über eine serielle UART-Verbindung (Universal Asynchronous Receiver/Transmitter). Der STM32 prüft die Datenintegrität, stellt Nutz- und Fehlerzustände auf einer Flüssigkristallanzeige (Liquid Crystal Display, LCD) dar und überwacht die Gesundheitszustände seiner Komponenten mit einem lokalen Supervisor.

Die Architektur kombiniert mehrere Schutzebenen: Protokollvalidierung, lokale Komponenten-Recovery, degradierte Betriebsmodi, getrennte systemd-Dienste mit Restart-Policy und Service-Watchdog, den Independent Watchdog (IWDG) des STM32 sowie den Hardware-Watchdog des Raspberry Pi. Die Zuordnung folgt dem Prinzip, die kleinste Recovery-Ebene zu verwenden, die den jeweiligen Fehler zuverlässig beseitigt. Untersucht wurden ein blockierter HTTPS-Wetterdatenpfad, GPS- und UART-Unterbrechungen, Dienst-Hänger und -Abstürze, ein STM32-Firmware-Hänger, fehlerhafte UART-Nachrichten, ein Displayausfall, Spannungsverlust, Unterspannung und ein Kernel-Hänger. Die jeweils fünfmal ausgeführten risikoarmen funktionalen Testfälle waren in allen Durchläufen erfolgreich. Beim blockierten HTTPS-Wetterdatenpfad, der keine Unterscheidung zwischen Netz- und Wetterdienstfehler erlaubt, betrugen Fehlerweitergabe und Recovery im Mittel 7,36 s beziehungsweise 10,10 s. Ein STM32-Firmware-Hänger wurde durch den IWDG in 5,70 s bis zur Rückkehr in den Anwendungsmodus behoben. Ein hängender GPS-Dienst benötigte im Mittel 28,46 s, ein abgestürzter Dienst dagegen 5,76 s. Nach vollständigem Spannungsverlust betrug die extern gemessene Netzwerk-Nichtverfügbarkeit im Mittel 32,33 s.

Der ergänzende Unterspannungstest am VIN-Eingang der NUCLEO-L432KC-Platine zeigte in fünf von fünf Durchläufen denselben stufenweisen Funktionsverlust: Normalbetrieb bei 8–6 V, verringerte LCD-Hintergrundbeleuchtung bei 5 V, Ausfall der Zeichendarstellung mit STM32-Neustart bei 4 V, vollständigen LCD-Ausfall mit erneutem Neustart bei 3 V und vollständigen Stillstand des STM32 bei 2 V. Beim Raspberry Pi belegte der Statuswert 0x50005 bei einer am Labornetzteil eingestellten Spannung von 4,50 V gleichzeitig aktive Unterspannung und Drosselung. Die Ergebnisse zeigen, dass eine abgestufte Fehlerbehandlung unnötige Gesamtneustarts vermeidet und in den durchgeführten Versuchen auch schwere Hänger behandeln konnte. Als Grenzen verbleiben insbesondere die manuelle Fehlerinjektion, das Hot-Start-Verhalten des GPS-Empfängers sowie die fehlende Messung der internen Versorgungsschienen und Reset-Ursachen während des Unterspannungstests.

Inhaltsverzeichnis

Zusammenfassung	I
Abbildungsverzeichnis	V
Tabellenverzeichnis	VI
Abkürzungsverzeichnis	VII
1 Einleitung	1
1.1 Motivation	1
1.2 Problemstellung	1
1.3 Zielsetzung	2
1.4 Forschungsfragen	2
1.5 Aufbau der Arbeit	2
2 Theoretische Grundlagen	3
2.1 Eingebettete Systeme und heterogene Fehlerdomänen	3
2.2 Dependability und Robustheit	3
2.3 Fault, Error und Failure	4
2.4 Fehlerbehandlung als Folge von Phasen	4
2.5 Recovery-orientierter Entwurf	5
2.6 Grundprinzip eines Watchdogs	5
2.7 Watchdog-Ebenen	5
2.7.1 Anwendungs- und Kommunikationsüberwachung	5
2.7.2 Service-Watchdog	6
2.7.3 Mikrocontroller-Watchdog	6
2.7.4 Systemweiter Hardware-Watchdog	6
2.8 Timeout-Dimensionierung	6
2.9 Heartbeat und Supervisor	7
2.10 Datenintegrität serieller Kommunikation	7
2.11 I2C und lokale Bus-Recovery	7
2.12 GPS, NMEA und externe Wetterdaten	8
2.13 Messung von Recovery-Zeiten	8
2.14 Zwischenfazit	9
3 Systementwurf und Implementierung	10
3.1 Anwendungsfall und Systemgrenze	10
3.2 Anforderungen und Fehlermodell	11
3.2.1 Funktionale Anforderungen	11
3.2.2 Nichtfunktionale Anforderungen	11
3.2.3 Fehlermodell	11
3.3 Gesamtarchitektur und Hardware	12
3.3.1 Ende-zu-Ende-Datenpfad und Verantwortungsgrenzen	13
3.3.2 Physischer Aufbau und Plattformgrenzen	13
3.4 Raspberry-Pi-Teilsystem	14
3.4.1 Gemeinsame Konfiguration und Zustandsaustausch	15
3.4.2 GPS-Erfassung	16
3.4.3 Wetterabruf und Abgrenzung der Fehlerursachen	16
3.4.4 Prozessüberwachung und Plattform-Recovery	17
3.5 UART als Plattformgrenze	18
3.5.1 Nachrichten, Integrität und Betriebsmodi	18
3.5.2 Sendeimplementierung auf dem Raspberry Pi	19
3.5.3 Empfang, Validierung und Resynchronisation auf dem STM32	19
3.6 STM32-Teilsystem	20
3.6.1 Firmwarestruktur, Bootablauf und Diagnose	20
3.6.2 LCD-Ausgabe und lokale I2C-Recovery	21

3.6.3	Supervisor, Gesundheitsmodell und IWDG	21
3.6.4	Spannungsüberwachung	22
3.7	Systemweites Recovery-Zusammenspiel	22
3.7.1	Eskalationshierarchie	22
3.7.2	Verteilter Recovery-Zustandsautomat	23
3.7.3	Diagnose und Nachvollziehbarkeit	25
3.8	Erfolgskriterien	25
4	Evaluation	26
4.1	Zielsetzung und Evaluationsfragen	26
4.2	Evaluationsaufbau und Beobachtungspunkte	26
4.2.1	Hardware und externe Instrumentierung	26
4.2.2	Softwareseitige Beobachtungspunkte	27
4.3	Messmethodik	28
4.3.1	Messgrößen	28
4.3.2	Monotone Zeitquelle und Journal-Cursor	28
4.3.3	Externe Messung bei vollständigen Neustarts	29
4.3.4	Manuell ausgelöste Hardwarefehler	29
4.3.5	Wiederholungen und statistische Kenngrößen	29
4.3.6	Dokumentation und Datenintegrität	29
4.4	Definition der Testfälle	30
4.5	Messergebnisse	31
4.5.1	Gesamtübersicht	31
4.5.2	N2: Blockierung und Wiederherstellung des HTTPS-Wetterdatenpfads	33
4.5.3	G1: Verlust und Wiederkehr des GPS-Empfängers	33
4.5.4	C1: Unterbrechung der Verbindung zum STM32	33
4.5.5	H1: Firmware-Hänger und STM32-IWDG	33
4.5.6	S1 und S2: Hängender gegenüber abgestürztem GPS-Dienst	34
4.5.7	S3: Hänger des UART-Dienstes	34
4.5.8	U1 und U2: UART-Datenintegrität und Puffergrenzen	35
4.5.9	L1: Lokale Recovery des LCD-/I2C-Pfads	35
4.5.10	P1: Vollständiger Spannungsverlust	35
4.5.11	P2: Kernel-Hänger und Hardware-Watchdog	36
4.5.12	P3: Kontrollierter Unterspannungstest	36
4.6	Zusammenfassung der Messergebnisse	38
5	Diskussion	39
5.1	Beantwortung der Forschungsfragen	39
5.1.1	F1: Strukturierung und geeignete Fehlerdomäne	39
5.1.2	F2: Eingriffstiefe und Wiederherstellungszeit	40
5.1.3	F3: Absturz, Hänger, Firmware-Hänger und System-Hänger	40
5.1.4	F4: Automatische Rückkehr zum definierten Zustand	40
5.2	Dienst-Hänger gegenüber Dienst-Absturz	40
5.3	Nutzen des degradierten Betriebs	41
5.4	Abgrenzung von Netzwerkpfad und Wetterdienst	41
5.5	Lokale Recovery und Zustandsverlust	41
5.6	Datenintegrität und Grenzen der XOR-Prüfsumme	42
5.7	Trade-off der Watchdog-Zeiten	42
5.8	Unterspannungsverhalten	42
5.9	Validität und Messunsicherheit	43
5.9.1	Interne Validität	43
5.9.2	Konstruktvalidität	43
5.9.3	Externe Validität	43
5.10	Gesamtbewertung	44
6	Fazit und Ausblick	45
6.1	Fazit	45

6.2 Ausblick	45
Literaturverzeichnis	47

Abbildungsverzeichnis

Abbildung 3.1 Gesamtarchitektur und wesentliche Datenpfade des Versuchssystems	10
Abbildung 3.2 Detailansicht des Raspberry-Pi-Teilsystems mit entkoppelten Diensten und Zustandsdateien	15
Abbildung 3.3 Vereinfachter Boot- und Recovery-Ablauf des STM32	21
Abbildung 3.4 Eskalationshierarchie der Recovery-Strategien	23
Abbildung 3.5 Systemweiter Zustandsautomat für Fehlerbehandlung und Recovery	24
Abbildung 4.1 Vergleich der mittleren funktionalen Wiederherstellungszeiten	34
Abbildung 4.2 Extern gemessene Ausfallzeiten der drei P1-Durchläufe	36
Abbildung 4.3 Qualitative Degradationsstufen in Abhängigkeit von der am Labornetzteil eingestellten VIN-Spannung	37
Abbildung 4.4 Raspberry-Pi-Status bei 4,50 V: throttled=0x50005	38

Tabellenverzeichnis

Tabelle 3.1 Funktionale Anforderungen	11
Tabelle 3.2 Nichtfunktionale Anforderungen	11
Tabelle 3.3 Fehlermodell und vorgesehene Recovery-Strategien	12
Tabelle 3.4 Stufen des Ende-zu-Ende-Datenpfads und ihre Verantwortungsgrenzen	13
Tabelle 3.5 Wesentliche Schnittstellen und Konfigurationswerte	14
Tabelle 3.6 Übergänge des systemweiten Recovery-Zustandsautomaten	25
Tabelle 4.1 Hardwarekomponenten des Versuchsaufbaus	27
Tabelle 4.2 Übersicht der untersuchten Testfälle	30
Tabelle 4.3 Zusammenfassung der zentralen Messergebnisse	32
Tabelle 4.4 Externe Netzwerknichtverfügbarkeit bei vollständigem Spannungsverlust	35
Tabelle 4.5 Reproduzierbares Unterspannungsverhalten der NUCLEO-L432KC- Platine	37
Tabelle 5.1 Zuordnung der untersuchten Fehlerklassen zu Recovery-Ebenen	39

Abkürzungsverzeichnis

ACK	Acknowledgement
API	Application Programming Interface
ASCII	American Standard Code for Information Interchange
BOR	Brown-out Reset
CDC	Communications Device Class
CMSIS	Cortex Microcontroller Software Interface Standard
CPU	Central Processing Unit
CRC	Cyclic Redundancy Check
CSV	Comma-Separated Values
DNS	Domain Name System
GND	Ground
GPIO	General Purpose Input/Output
HSI	High-Speed Internal Oscillator
HTTP(S)	Hypertext Transfer Protocol (Secure)
I2C	Inter-Integrated Circuit
ICMP	Internet Control Message Protocol
I/O	Input/Output
IWDG	Independent Watchdog
JSON	JavaScript Object Notation
LCD	Liquid Crystal Display
LSI	Low-Speed Internal Oscillator
NACK	Not Acknowledged
NMEA	National Marine Electronics Association
NTP	Network Time Protocol
PID	Process Identifier
PVD	Programmable Voltage Detector
RCC	Reset and Clock Control
RMC	Recommended Minimum Specific GNSS Data
RTC	Real-Time Clock
RX/TX	Receive/Transmit
SCL	Serial Clock
SDA	Serial Data
SHA-256	Secure Hash Algorithm 256 Bit
TCP	Transmission Control Protocol
TLS	Transport Layer Security
UART	Universal Asynchronous Receiver/Transmitter
USB	Universal Serial Bus
VIN	Voltage Input
WCET	Worst-Case Execution Time
WDT	Watchdog Timer
WMO	World Meteorological Organization
XOR	Exclusive OR

1 Einleitung

1.1 Motivation

Eingebettete Systeme übernehmen klar abgegrenzte Funktionen innerhalb technischer Anlagen und arbeiten häufig lange ohne unmittelbare menschliche Überwachung. Ihre Verfügbarkeit hängt nicht allein von der eigentlichen Anwendungslogik ab. Sensoren und Aktoren, Kommunikationspfade, Betriebssystemdienste, externe Server und die Energieversorgung bilden zusätzliche Fehlerdomänen. Eine Implementierung, die nur den Normalfall berücksichtigt, kann bereits durch einen vorübergehenden Netzwerkausfall oder einen blockierten Prozess dauerhaft unbrauchbar werden. Robustheit bedeutet in diesem Zusammenhang nicht, Fehler vollständig zu verhindern. Entscheidend ist, dass Fehler erkannt, eingegrenzt und mit einer zur Fehlerklasse passenden Maßnahme behandelt werden. Die Dependability-Taxonomie unterscheidet unter anderem Verfügbarkeit, Zuverlässigkeit, Integrität und Wartbarkeit als Eigenschaften, die Vertrauen in ein System begründen [1, S. 13–14]. Recovery-orientierte Ansätze verschieben den Entwurfsfokus zusätzlich von der ausschließlichen Vermeidung von Fehlern hin zu einer schnellen und überprüfbaren Wiederherstellung [2, S. 1]. Ein pauschaler Neustart des Gesamtsystems ist zwar einfach, aber nicht für jede Störung angemessen. Ein ungültiger UART-Datenrahmen kann lokal verworfen werden. Ein nicht kritischer Displayausfall kann zu einem degradierten Betrieb führen, ohne die übrige Funktion zu unterbrechen. Ein Firmware-Hänger des Mikrocontrollers erfordert dagegen einen Reset, während ein Kernel-Hänger des Linux-Systems nur durch eine vom betroffenen Softwarepfad unabhängige Instanz behoben werden kann. Ein Watchdog überwacht, ob die zugeordnete Funktion innerhalb einer Frist ein gültiges Fortschrittszeichen liefert. Als Recovery wird in dieser Arbeit die Rückkehr in einen definierten funktionsfähigen Zustand bezeichnet; ein degradierter Betrieb erhält dabei die Kernfunktion mit bewusst eingeschränktem Funktionsumfang. Daraus entsteht die Notwendigkeit einer abgestuften Watchdog- und Recovery-Architektur.

1.2 Problemstellung

Die zentrale Problemstellung besteht darin, geeignete Überwachungs- und Wiederherstellungsmechanismen für unterschiedliche Fehlerklassen in einem heterogenen eingebetteten System auszuwählen und ihre Wirksamkeit experimentell zu bewerten. Das betrachtete System umfasst einen Raspberry Pi mit Linux, einen STM32-Mikrocontroller, einen Binghe VK-162 USB-GPS-Empfänger, eine Internetverbindung und ein LCD. Fehler können daher auf Daten-, Komponenten-, Prozess-, Firmware- und Systemebene auftreten. Die technische Herausforderung liegt nicht nur im Aktivieren einzelner Watchdogs. Ein Prozessabsturz ist für den Service-Manager unmittelbar sichtbar, während ein weiterhin vorhandener, aber hängender Prozess erst durch ausbleibende Fortschrittsnachweise erkannt wird. Ein Watchdog-Lebenszeichen darf

zudem nicht bedingungslos erzeugt werden, weil sonst ein funktionierender Timer einen Stillstand der eigentlichen Nutzfunktion verdecken könnte. Schließlich soll eine Recovery möglichst nur die tatsächlich betroffene Fehlerdomäne zurücksetzen.

1.3 Zielsetzung

Ziel dieser Arbeit ist die Untersuchung, Implementierung und experimentelle Bewertung mehrerer Watchdog- und Recovery-Strategien für ein heterogenes System aus Raspberry Pi und STM32. Als Testanwendung bestimmt der Raspberry Pi eine Position über GPS, ruft Wetterdaten ab und überträgt den Zustand an den STM32, der die Daten auf einem LCD darstellt. Die Wetteranwendung ist Mittel zum Zweck: Sie erzeugt einen nachvollziehbaren Datenpfad mit realen externen Abhängigkeiten und erlaubt die gezielte Fehlerinjektion, also das kontrollierte Herbeiführen eines zuvor festgelegten Fehlers. Untersucht werden die Auswahl der Recovery-Ebene, die Erkennungs- und Wiederherstellungszeiten sowie die Frage, ob ein vorab durch messbare Kriterien definierter funktionaler Endzustand ohne manuellen Neustart erneut erreicht wird.

1.4 Forschungsfragen

F1: Wie muss ein heterogenes eingebettetes System strukturiert werden, damit Fehler auf der jeweils geeigneten Ebene erkannt und behandelt werden?

F2: Wie unterscheiden sich lokale Recovery, Dienstneustart, Mikrocontroller-Reset und vollständiger Systemneustart hinsichtlich Eingriffstiefe und funktionaler Wiederherstellungszeit?

F3: Welche Unterschiede zeigen sich zwischen Prozessabsturz, Prozess-Hänger, Firmware-Hänger und vollständigem Linux-Hänger?

F4: In welchem Umfang kann das entwickelte System nach den betrachteten Hardware-, Kommunikations- und Softwarefehlern automatisch in einen definierten Zustand zurückkehren?

1.5 Aufbau der Arbeit

Kapitel 2 führt Dependability, Watchdogs, Supervisoren und Recovery-Strategien ein. Kapitel 3 verbindet Anforderungen und Fehlermodell unmittelbar mit dem Systementwurf und der komponentenorientierten Implementierung auf Raspberry Pi und STM32. Kapitel 4 dokumentiert Fehlerinjektion, Messmethodik und Ergebnisse. Kapitel 5 ordnet die Ergebnisse ein und beantwortet die Forschungsfragen. Kapitel 6 fasst die Erkenntnisse zusammen und nennt Erweiterungsmöglichkeiten.

2 Theoretische Grundlagen

2.1 Eingebettete Systeme und heterogene Fehlerdomänen

Ein eingebettetes System ist in eine technische Umgebung integriert und stellt eine spezialisierte Funktion bereit. In einem heterogenen Aufbau können ein Linux-Rechner, ein Mikrocontroller, Kommunikationsschnittstellen und externe Dienste gleichzeitig beteiligt sein. Diese Teile besitzen unterschiedliche Laufzeitmodelle und Fehlermechanismen. Ein Linux-Dienst kann abstürzen oder blockieren, ein Mikrocontroller kann in einer Endlosschleife verbleiben, eine serielle Verbindung kann Daten verlieren und ein externer Dienst kann zeitweise nicht erreichbar sein. Für die Fehlerbehandlung ist deshalb die Definition von Fehlerdomänen hilfreich. Eine Fehlerdomäne umfasst Komponenten, die durch dieselbe Fehlerursache gemeinsam ausfallen können. Ein Software-Watchdog im Linux-Prozess ist gegenüber Kernel-Hängern nicht unabhängig [3, Abschn. „The simplest API“]. Der unabhängige Hardware-Watchdog des STM32 (Independent Watchdog, IWDG) arbeitet dagegen auch bei einem Stillstand der Hauptschleife weiter, da ihn ein separater interner Niedriggeschwindigkeitsoszillator (Low-Speed Internal Oscillator, LSI) taktet [4, S. 1037-1045, Abschn. 33].

2.2 Dependability und Robustheit

Avizienis et al. fassen unter Dependability Eigenschaften zusammen, die ein begründetes Vertrauen in einen Dienst ermöglichen. Dazu gehören Verfügbarkeit, Zuverlässigkeit, Safety, Integrität und Wartbarkeit [1, S. 13-14]. Verfügbarkeit bezeichnet die Bereitschaft zur korrekten Dienstleistung, Zuverlässigkeit deren Kontinuität, Safety die Abwesenheit unvermeidbarer Folgen, Integrität die Abwesenheit unzulässiger Zustandsänderungen und Wartbarkeit die Fähigkeit zur Durchführung von Reparaturen oder Änderungen. Für die vorliegende Arbeit wird Robustheit operational durch folgende Eigenschaften beschrieben:

1. Ein Fehler wird in einem nachvollziehbaren Zeitfenster erkannt.
2. Der Fehler wird in einen definierten internen und extern sichtbaren Zustand übersetzt.
3. Nicht betroffene Komponenten bleiben nach Möglichkeit aktiv.
4. Eine zur Fehlerklasse passende Recovery-Maßnahme wird ausgeführt.
5. Die Rückkehr zu einer funktionsfähigen Konfiguration ist durch Logs, Statusdateien oder Diagnosemeldungen nachweisbar. Robustheit ist damit nicht gleichbedeutend mit vollständiger Fehlertransparenz. Eine ausdrücklich als nicht verfügbar gekennzeichnete Wetterinformation kann robuster sein als die unmarkierte Darstellung veralteter Werte, weil die Datenqualität korrekt kommuniziert wird.

2.3 Fault, Error und Failure

Die Dependability-Taxonomie unterscheidet eine Kette aus Fault, Error und Failure [1, S. 13 und 20–22]. Ein Fault ist die Ursache, etwa eine unterbrochene Busleitung oder ein blockierter Prozess. Der Fault kann einen fehlerhaften internen Zustand erzeugen, beispielsweise eine abgelehnte Busübertragung oder einen veralteten gespeicherten Zustand. Wird die geforderte Funktion an der Systemgrenze nicht mehr erbracht, liegt ein Failure vor. Diese Trennung ist für die Evaluation wesentlich. Das Abziehen des GPS-Empfängers ist die Fehlerinjektion. Der daraus entstehende Verlust einer aktuell gültigen Position ist ein interner Fehlerzustand. Die ausdrücklich gekennzeichnete Nutzung der letzten noch zulässigen Position ist dagegen eine kontrollierte Reaktion, die einen unmarkierten Ausfall der Gesamtfunktion verhindert. Die konkreten Zustandsfelder des Versuchsprotokolls werden in Abschnitt 3.5 eingeführt.

Unterspannung, Brown-out und intermittierende Versorgung bilden eine eigene Fehlerklasse. Ein Brown-out ist kein zwingend vollständiger Spannungsverlust, sondern ein Betriebszustand, in dem die Versorgung unter den für zuverlässige Ausführung erforderlichen Bereich absinkt. Balsamo et al. unterscheiden bei intermittierender Versorgung eine minimale Betriebsspannung sowie getrennte Schwellen für ein gesichertes Ruhestadium (Hibernation) und die Wiederaufnahme; unterhalb der minimalen Spannung kann ein Prozessor unvorhersehbar arbeiten oder vollständig abschalten [26, S. 15–16]. Ein Brown-out Reset (BOR) ist dagegen die hardwareseitige Reaktion des Mikrocontrollers auf das Unterschreiten einer internen Schwelle. Ein beobachteter Neustart bei einer am Netzteil eingestellten Spannung ist deshalb nicht automatisch mit dieser BOR-Schwelle gleichzusetzen.

Forschung zu intermittierend versorgten eingebetteten Systemen zeigt außerdem, dass wiederholte Spannungsausfälle nicht nur Verfügbarkeit, sondern auch Zustandskonsistenz betreffen. Mementos misst die verfügbare Energie und sichert den Ausführungszustand vor einem erwarteten Spannungsverlust in nichtflüchtigem Speicher [27, S. 159–163]. DINO zeigt, dass Neustarts an beliebigen Programmpunkten zu teilweise oder wiederholt ausgeführten Operationen und damit zu inkonsistenten Zuständen führen können [28, S. 575–577]. Die vorliegende Arbeit implementiert keine solche Sicherung eines später wiederaufnehmbaren Ausführungszustands (Checkpointing); die Arbeiten dienen als theoretische Grundlage für die Trennung von Spannungserkennung, definiertem Zustand, Neustart und anschließender Recovery.

2.4 Fehlerbehandlung als Folge von Phasen

Eine vollständige Fehlerbehandlung besteht aus mehreren Phasen:

Erkennung: Ein Timeout, eine Ausnahme, ein ungültiger Datenrahmen oder ein Hardwarestatus signalisiert eine Abweichung.

Eingrenzung: Die betroffene Fehlerdomäne wird bestimmt, damit funktionierende Komponenten nicht unnötig zurückgesetzt werden.

Bewertung: Kritikalität, Alter des Zustands und Anzahl aufeinander folgender Fehler werden berücksichtigt.

Recovery: Retry, Reinitialisierung, Dienstneustart, Reset oder Reboot stellen einen definierten Zustand her.

Verifikation: Die ursprüngliche Funktion wird erneut geprüft; ein Prozess gilt nicht bereits durch seine Existenz als funktional. Die Phasenfolge wird in dieser Arbeit als projektspezifische Struktur verwendet. Sie knüpft an Randells Trennung von Fehlererkennung, Zustandswiederherstellung und alternativer Ausführung an [5, S. 220-223].

2.5 Recovery-orientierter Entwurf

Recovery-Oriented Computing betrachtet Hardwarefehler, Softwarefehler und Bedienfehler als unvermeidbare Ereignisse und richtet den Entwurf stärker auf eine kurze mittlere Wiederherstellungszeit (Mean Time to Repair) aus [2, S. 1 und 5]. Dazu gehören gezielte Recovery-Experimente, die Partitionierung von Fehlerdomänen, mehrere möglichst unabhängige Schutzebenen (Defense in Depth) und die Beobachtbarkeit von Reparaturmechanismen. Die Idee feingranularer Recovery wird durch Microreboots konkretisiert: Anstatt eine gesamte Anwendung zurückzusetzen, wird möglichst nur die fehlerhafte Komponente neu gestartet [6, S. 31–32]. Dieses Prinzip lässt sich auf die vorliegende Arbeit übertragen. Ein abgestürzter GPS-Dienst wird einzeln neu gestartet; ein LCD-Fehler wird lokal behandelt; ein vollständiger Raspberry-Pi-Neustart bleibt die letzte Eskalationsstufe.

2.6 Grundprinzip eines Watchdogs

Ein Watchdog erwartet ein periodisches Lebenszeichen innerhalb eines vorgegebenen Zeitfensters. Bleibt das Lebenszeichen aus, löst er eine vordefinierte Aktion aus [7, Abschn. WatchdogSec=, 8, Abschn. WATCHDOG=1, 4, S. 1037–1045, Abschn. 33]. Das Lebenszeichen muss den Fortschritt der überwachten Funktion repräsentieren. Wird ein Hardware-Watchdog bedingungslos in einem Timer-Interrupt aktualisiert, kann eine blockierte Hauptschleife unentdeckt bleiben [11, S. 6-7]. Ein Watchdog erkennt primär Zeitüberschreitungen, nicht automatisch die semantische Korrektheit. Eine fehlerhafte, aber weiterhin schnell laufende Schleife kann den Timer fristgerecht bedienen. Deshalb ergänzt auf dem STM32 ein Supervisor, also eine übergeordnete Logik zur Zusammenführung mehrerer Gesundheitszustände, die reine Zeitüberwachung.

2.7 Watchdog-Ebenen

2.7.1 Anwendungs- und Kommunikationsüberwachung

Auf Anwendungsebene werden Ausnahmen, Zustandsalter, Prüfsummen und periodische Lebenszeichen zwischen Komponenten verwendet, die im Folgenden als Heartbeats bezeichnet werden. Diese Mechanismen können differenzierte Fehlerzustände erzeugen und lokale Recovery auslösen. Sie fallen jedoch aus, wenn die gesamte Ausführungsumgebung blockiert.

2.7.2 Service-Watchdog

Ein Service-Manager kann Prozesse überwachen. Bei systemd aktiviert WatchdogSec= einen Dienst-Watchdog. Der Dienst muss periodisch WATCHDOG=1 über die systemd-Benachrichtigungsschnittstelle sd_notify() senden; bleibt die Nachricht aus, behandelt der Service-Manager den Dienst als fehlerhaft [7, Abschn. WatchdogSec= und Restart=, 8, Abschn. WATCHDOG=1]. In Verbindung mit Restart= entsteht eine Recovery für Prozesse, die formal existieren, aber keinen Fortschritt mehr machen.

2.7.3 Mikrocontroller-Watchdog

Der STM32L432KC besitzt einen Independent Watchdog, der aus dem Low-Speed-Internal-Oszillator getaktet wird und einen Reset auslöst, wenn der Zähler nicht rechtzeitig aktualisiert wird [4, S. 1037–1045, Abschn. 33, 9, S. 30 und 41]. Die unabhängige Taktquelle reduziert gemeinsame Fehlerursachen mit dem Haupttakt. Ein Window Watchdog kann darüber hinaus auch ein zu frühes Lebenszeichen als Fehler erkennen; er wurde in der Arbeit nicht verwendet.

2.7.4 Systemweiter Hardware-Watchdog

Linux stellt Hardware-Watchdogs über eine Programmierschnittstelle (Watchdog-API) und Geräte wie /dev/watchdog0 bereit [3, Abschn. „The simplest API“]. Solange ein Prozess im Benutzerraum (Userspace) den Timer innerhalb des vorgegebenen Zeitfensters aktualisiert, bleibt das System aktiv. Bei einem schweren Hänger läuft der Timer ab und die Hardware löst einen Neustart aus. Diese Ebene schützt gegen Fehler, die der systemd-Service-Manager als Prozess mit der Prozess-ID 1 (PID 1) nicht mehr behandeln kann.

2.8 Timeout-Dimensionierung

Die Dimensionierung eines Watchdog-Timeouts ist zunächst ein allgemeines Entwurfsproblem. Der Timeout muss größer sein als der größte im fehlerfreien Betrieb erwartete Abstand zwischen zwei gültigen Fortschrittsmeldungen. Zusätzlich sind zeitliche Schwankungen durch Scheduling, Interrupts und asynchrone Kommunikation sowie eine Sicherheitsreserve zu berücksichtigen [10, S. 4 und 12] [11, S. 16].

Dabei bezeichnet T_{WDT} den Watchdog-Timeout, $T_{\text{max, normal}}$ den größten erwarteten Abstand zwischen zwei zulässigen Fortschrittsmeldungen, T_{Jitter} die zusätzliche zeitliche Schwankung und T_{Reserve} eine Sicherheitsmarge. Solche periodischen Fortschrittsmeldungen werden gegenüber einem Watchdog auch als Keepalives bezeichnet. Daraus ergibt sich als vereinfachte Entwurfsregel:

$$T_{\text{WDT}} \geq T_{\text{max, normal}} + T_{\text{Jitter}} + T_{\text{Reserve}} \quad (2.1)$$

Ein weiterer methodischer Kritikpunkt ist, dass die vereinfachte Beziehung lediglich eine Untergrenze für den Watchdog-Timeout beschreibt. Damit ein Fehler noch innerhalb der für das System zulässigen Reaktionszeit erkannt und behandelt wird, ist zusätzlich eine anforderungsseitige Obergrenze festzulegen; sie muss auch Reset-, Boot- und Dienstanlaufzeiten berücksichtigen. Außerdem sind $T_{\text{max, normal}}$ und T_{Jitter}

nur dann additiv, wenn der erste Term den maximalen nominalen Keepalive-Abstand ohne die separat erfassten Schwankungen bezeichnet. Wird er dagegen aus den im Normalbetrieb beobachteten maximalen Abständen bestimmt, können Scheduling-, Interrupt- und Kommunikationsverzögerungen bereits enthalten sein; ihre erneute Addition würde dieselben Einflüsse teilweise doppelt berücksichtigen. Die Größen müssen daher durch voneinander abgegrenzte Mess- oder Analysegrenzen definiert werden.

Damit bleibt die Beziehung eine Entwurfsheuristik und kein formaler Nachweis der maximalen Ausführungszeit (Worst-Case Execution Time, WCET). Ein langer Timeout reduziert Fehllarme, verlängert aber die Reaktion auf echte Hänger; ein kurzer Timeout reagiert schneller, kann bei hoher Last oder blockierender Ein-/Ausgabe (Input/Output, I/O) unnötige Neustarts auslösen. Die konkreten Timeout-Werte des aufgebauten Systems werden in Abschnitt 3.4.1 festgelegt.

2.9 Heartbeat und Supervisor

Ein Heartbeat ist ein Lebenszeichen zwischen Komponenten. Beim PING/PONG-Verfahren dieser Arbeit sendet der Raspberry Pi PING; der STM32 muss die Nachricht empfangen, verarbeiten und innerhalb des konfigurierten Timeouts mit PONG antworten. Ein Supervisor aggregiert mehrere Gesundheitsinformationen. Neben einem Zeitstempel kann er Kritikalität, Fehlerbestätigungen, Recovery-Zähler und den globalen Systemmodus speichern. Mehrfachabtastungen vermeiden, dass ein einzelner transienter Fehler sofort zu einer Zustandsänderung führt. Der STM32 bestätigt LCD-Fehler und -Wiederherstellungen jeweils erst nach mehreren aufeinander folgenden Messungen. Kritische Subsysteme beeinflussen die Freigabe des IWDG-Refresh; bei einem nicht kritischen Displayfehler bleibt die Kernfunktion im eingeschränkten Modus DEGRADED verfügbar.

2.10 Datenintegrität serieller Kommunikation

UART stellt zwischen Raspberry Pi und STM32 eine asynchrone serielle Verbindung bereit und liefert einen Bytestrom ohne inhärente Rahmen- oder Integritätsgarantie. Ein Anwendungsprotokoll benötigt deshalb eine Rahmengrenze, eine maximale Länge und eine Prüfinformation. Die Arbeit verwendet eine ASCII-codierte Textzeile mit einer XOR-Prüfsumme, bei der die Nutzdatenbytes durch die bitweise Operation Exklusiv-Oder zusammengeführt werden. Dieses Verfahren ist einfach und schnell, erkennt aber deutlich weniger Fehlermuster als gut ausgewählte zyklische Redundanzprüfungen (Cyclic Redundancy Check, CRC) [12, S. 59–60 und 67–69, 13, S. 145–146].

2.11 I2C und lokale Bus-Recovery

I2C ist ein synchroner Zweidrahtbus mit der Taktleitung SCL (Serial Clock) und der Datenleitung SDA (Serial Data). Seine Open-Drain-Ausgänge ziehen die Leitungen nur aktiv auf den Low-Pegel; der High-Pegel entsteht über Pull-up-Widerstände. Die

offizielle Spezifikation beschreibt unter anderem START/STOP-Bedingungen, Empfangsbestätigungen, eine negative Bestätigung (NACK) und das Bus-Clear-Verfahren [14, Abschn. 3.1.1, 3.1.4-3.1.6, 3.1.16]. Bleibt eine Peripherie oder Leitung in einem fehlerhaften Zustand, darf das BUSY-Statusbit die Anwendung nicht dauerhaft in einer Warteprüfung halten. Die Arbeit begrenzt deshalb Wartezeiten, setzt den I2C-Block zurück und erzeugt bei Bedarf Taktpulse auf SCL zur Busfreigabe, bevor das LCD neu initialisiert wird. Das angeschlossene LCD2004-Zeichenmodul wird über den Adapter als HD44780-basiertes Display angesteuert [19, S. 4]. Der eingesetzte AZDelivery-I2C-Adapter verwendet einen PCF8574-Ein-/Ausgabe-Expander, der die seriell übertragenen I2C-Daten auf parallele LCD-Signale umsetzt [19, S. 4]. Die Eigenschaften und der hardwareseitig einstellbare Adressbereich des Expanders sind im NXP-Datenblatt dokumentiert [15, S. 1 und 5-8].

2.12 GPS, NMEA und externe Wetterdaten

Als vollständiges USB-GPS-Gerät wurde ein unter der Vertriebsbezeichnung Binghe VK-162 erworbenes Modul eingesetzt. Das Gerät erschien unter Linux als serieller Gerätepfad `/dev/ttyACM0`. Seine interne Empfängereinheit identifizierte sich in den protokollierten USB- und Startmeldungen als Hardware der u-blox-7-Familie beziehungsweise als UBX-G70xx; diese Bezeichnung beschreibt die intern erkannte Einheit und nicht die Vertriebsbezeichnung des vollständigen USB-Geräts. Die experimentelle Identifikation ist in Anhang A dokumentiert. Die interne Empfängereinheit liefert über USB zeilenorientierte NMEA-Textdatensätze für Navigationsinformationen. Der verwendete Satztyp RMC enthält Zeit, Gültigkeitsstatus, Position und Bewegungsinformationen; nur der Status A kennzeichnet gültige Navigationsdaten [16, S. 62]. Die Umrechnung des in Grad und Minuten angegebenen Breiten- oder Längengrads φ in Dezimalgrad erfolgt nach

$$\varphi_{\text{dez}} = \varphi_{\text{Grad}} + \frac{\varphi_{\text{Minuten}}}{60} \quad (2.2)$$

Dabei bezeichnet φ_{Grad} den ganzzahligen Gradanteil und φ_{Minuten} den Minutenanteil. Die Formel liefert zunächst den Betrag; die Richtungsangaben Süd beziehungsweise West erhalten anschließend ein negatives Vorzeichen, Nord und Ost ein positives.

Der Wetterdienst wird über eine HTTPS-Schnittstelle mit einer Antwort im Datenformat JSON angesprochen [17, Abschn. „Weather Forecast API“]. Als externe Abhängigkeit ist er bewusst Teil des Fehlermodells: Netzwerkkonnektivität, die Namensauflösung über das Domain Name System (DNS), der abgesicherte Verbindungsaufbau über Transport Layer Security (TLS) und die Verfügbarkeit beziehungsweise Antwort des Wetterdienstes können unabhängig voneinander ausfallen.

2.13 Messung von Recovery-Zeiten

Für Zeitdifferenzen innerhalb eines laufenden Linux-Systems ist eine monotone Zeitquelle erforderlich, weil die Echtzeituhr durch eine Netzwerkzeitsynchronisation

mittels Network Time Protocol (NTP) oder einen manuellen Abgleich springen kann. Die Testskripte verwenden daher `time.monotonic()`. Bei vollständigen Neustarts ist eine externe Messinstanz notwendig, da der zu untersuchende Rechner während des Ausfalls keine kontinuierliche Zeitreihe schreiben kann. Bei kleinen Stichproben werden Mittelwert, Median, Minimum, Maximum und Stichproben-Standardabweichung berichtet.

2.14 Zwischenfazit

Keine einzelne Watchdog-Ebene deckt alle Fehlerklassen ab. Datenfehler, Peripheriefehler, Prozess-Hänger, Firmware-Hänger und Kernel-Hänger benötigen unterschiedliche Erkennungs- und Recovery-Mechanismen. Daraus wird im folgenden Kapitel eine hierarchische Architektur abgeleitet, die lokal beginnt und nur bei schwerwiegenden Fehlern bis zum vollständigen Systemneustart eskaliert.

3 Systementwurf und Implementierung

Dieses Kapitel führt Anforderungen, Entwurfsentscheidungen und deren konkrete Umsetzung in einer gemeinsamen Struktur zusammen. Die Darstellung folgt dabei zwei gekoppelten Pfaden: dem Nutzdatenpfad von der Positionserfassung bis zur LCD-Ausgabe und dem Überwachungs- und Recoverypfad von der lokalen Fehlererkennung bis zum Plattform-Reset. Für jedes Teilsystem werden Rolle, Schnittstellen, Implementierung, Gesundheitskriterien und Eskalationsgrenze gemeinsam beschrieben.

3.1 Anwendungsfall und Systemgrenze

Als Testumgebung wird ein eingebettetes System verwendet, das seine Position über einen Binghe VK-162 USB-GPS-Empfänger bestimmt, Wetterdaten für diese Position von einem externen Dienst abrufen und die Informationen auf einem LCD darstellt. Der Raspberry Pi übernimmt netzwerk- und betriebssystemnahe Aufgaben. Der STM32 übernimmt die hardwarenahe Displaysteuerung und eine unabhängige Überwachung der Mikrocontroller-Firmware. Der Datenfluss umfasst GPS-Erfassung, Wetterabfrage, UART-Übertragung und Anzeige. Die Fehlerdomänen und Schnittstellen sind in Abbildung 3.1 dargestellt.

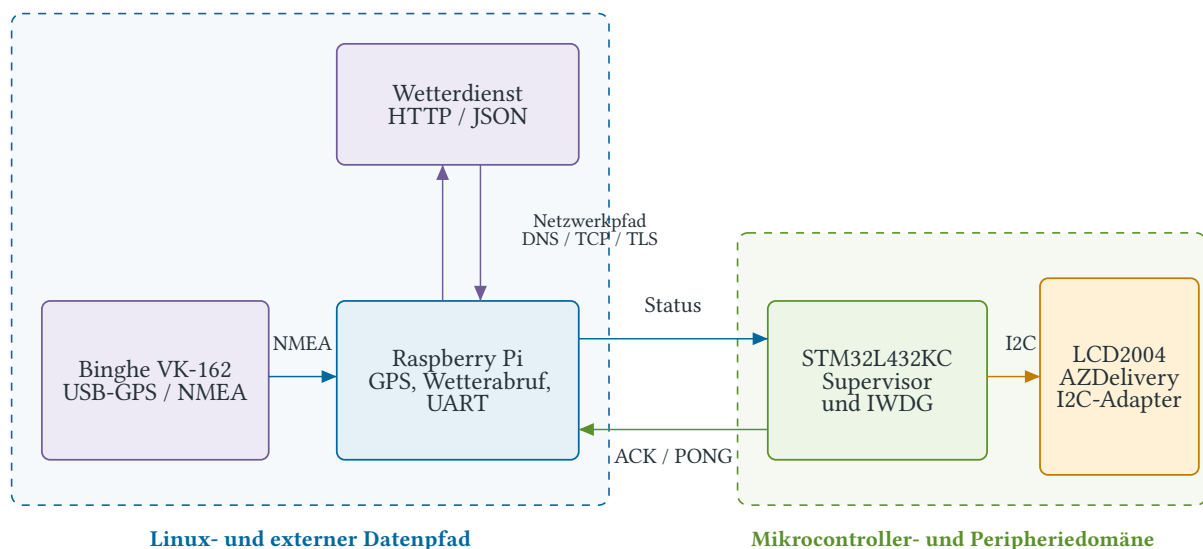


Abbildung 3.1: Gesamtarchitektur und wesentliche Datenpfade des Versuchssystems

Der Anwendungsfall wurde gewählt, weil er mehrere voneinander unabhängige Fehlerquellen in einem überschaubaren Aufbau vereint. Die wissenschaftliche Fragestellung betrifft nicht das Anzeigen von Wetterdaten, sondern die Überwachung und Wiederherstellung des vollständigen Datenpfads.

3.2 Anforderungen und Fehlermodell

Die Anforderungen definieren zunächst die erwartete Nutzfunktion und die für eine robuste Ausführung maßgeblichen Qualitätsmerkmale. Das anschließende Fehlermodell ordnet diesen Anforderungen konkrete Fehlerklassen, Erkennungsmechanismen und Recovery-Maßnahmen zu.

3.2.1 Funktionale Anforderungen

Tabelle 3.1: Funktionale Anforderungen

ID	Anforderung
FA-01	Das System muss eine gültige GPS-Position erfassen können.
FA-02	Das System muss Wetterdaten für die zuletzt gültige Position abrufen.
FA-03	Der Raspberry Pi muss den GPS-Zustand und die Verfügbarkeit eines gültigen Wetterdatensatzes an den STM32 übertragen.
FA-04	Der STM32 muss gültige Datenrahmen prüfen und auf dem LCD darstellen.
FA-05	Der Raspberry Pi und der STM32 müssen ihre Verbindung über einen Heartbeat prüfen.
FA-06	Fehlerzustände müssen intern protokolliert und gegenüber nachgelagerten Komponenten gekennzeichnet werden.
FA-07	Nach Wegfall einer Störung muss der Normalbetrieb automatisch wiederhergestellt werden.

3.2.2 Nichtfunktionale Anforderungen

Tabelle 3.2: Nichtfunktionale Anforderungen

ID	Anforderung
NFA-01	Ein Fehler einer Teilfunktion darf andere funktionsfähige Teilfunktionen nicht unnötig stoppen.
NFA-02	Die Recovery muss ohne manuellen Eingriff erfolgen, sofern die Fehlerursache nicht physisch bestehen bleibt.
NFA-03	Nicht kritische Fehler sollen lokal behandelt werden, bevor ein Reset eskaliert wird.
NFA-04	UART-Datenrahmen müssen gegen ungültige Prüfsummen und Pufferüberläufe abgesichert werden.
NFA-05	Hängende Prozesse müssen von abgestürzten Prozessen unterschieden und erkannt werden.
NFA-06	Mess- und Diagnoseinformationen müssen reproduzierbar protokolliert werden.
NFA-07	Die Watchdog-Aktualisierung darf nicht erfolgen, wenn die überwachte Funktion keinen Fortschritt mehr macht.

3.2.3 Fehlermodell

Das Fehlermodell umfasst Fehler auf Daten-, Komponenten-, Prozess-, Firmware- und Systemebene. Für jede Fehlerklasse werden Erkennung und Recovery bereits im Systementwurf festgelegt.

Tabelle 3.3: Fehlermodell und vorgesehene Recovery-Strategien

Fehler	Ebene	Erkennung	Recovery
Ungültige UART-Prüfsumme	Daten	Prüfsummenvergleich	Rahmen verwerfen, Fehlerantwort senden
UART-Pufferüberlauf	Daten/Link	Längenbegrenzung	Pufferzustand verwerfen und Synchronisation fortsetzen
Netzwerkpfad nicht verfügbar	Netzwerk	Link-, Routing-, DNS- oder TLS-Diagnose	Retry mit Backoff; Netzschnittstelle nur bei bestätigtem lokalem Fehler neu initialisieren
Wetterdienst oder Antwort fehlerhaft	Externer Dienst/Daten	HTTP-Status und Antwortvalidierung	Wetterdaten als nicht verfügbar markieren; Retry mit Backoff ohne Netzwerkreset
Kein GPS-Gerät oder kein Fix	Peripherie	Geräte-/Timeout-Prüfung	Letzte gültige Daten verwenden, MODE=LAST
UART-Verbindung unterbrochen	UART-Verbindung	PING/PONG-Timeout	Wiederholungsversuch und automatische Wiederherstellung der Verbindung
LCD getrennt	I2C/LCD	NACK oder Schreibfehler	Lokale LCD-/I2C-Recovery, Modus DEGRADED
Linux-Dienst abgestürzt	Prozess	Prozessende durch systemd	Automatischer Dienstneustart
Linux-Dienst hängt	Prozess	Ausbleibendes systemd-Watchdog-Signal	Prozess beenden und Dienst neu starten
STM32-Firmware hängt	Firmware	IWDG-Timeout	Mikrocontroller-Reset und Bootdiagnose
Linux-Kernel hängt	Linux	BCM2835-Hardware-Watchdog	Vollständiger Neustart des Raspberry Pi
Versorgung fällt aus	System	Neustart nach Rückkehr der Versorgung	Automatischer Boot und Start aller Dienste

Allgemeine Netzwerkkonnektivität und Verfügbarkeit des Wetterdienstes sind unterschiedliche Fehlerquellen. Das Fehlermodell trennt sie deshalb einschließlich der jeweils angemessenen Recovery. Der Prototyp erfasst diese Ursachen jedoch nicht unabhängig: Ein fehlgeschlagener Abruf wird in beiden Fällen auf denselben Status abgebildet. Dieser Status belegt nur, dass aktuell kein gültiger Wetterdatensatz vorliegt; er ist kein Nachweis für einen Ausfall des Netzwerks. Die Implementierungsgrenze wird in Abschnitt 3.4.3 erläutert.

3.3 Gesamtarchitektur und Hardware

Die Gesamtarchitektur wird aus zwei gekoppelten Perspektiven beschrieben. Der Nutzdatenpfad führt von der Positionserfassung über Wetterabfrage und UART-Übertragung bis zur LCD-Ausgabe. Parallel dazu bewertet der Überwachungs- und

Recoverypfad Lebenszeichen, Kommunikationszustände und lokale Fehler. Er hält eine Recovery möglichst innerhalb der betroffenen Komponente und eskaliert erst, wenn diese lokale Behandlung nicht ausreicht. Auf diese Weise werden die in Abschnitt 3.2.3 eingeführten Fehlerklassen den Stufen des Datenpfads und ihren zuständigen Recovery-Ebenen zugeordnet.

3.3.1 Ende-zu-Ende-Datenpfad und Verantwortungsgrenzen

Die Gesamtfunktion entsteht in vier aufeinanderfolgenden Stufen: Der GPS-Dienst erzeugt zunächst einen Positionszustand. Der Wetterabrufdienst reichert ihn mit Wetterdaten an. Der UART-Dienst verdichtet beide Zustände zu einem seriellen Rahmen und überträgt ihn über die Plattformgrenze. Der STM32 prüft den Rahmen, aktualisiert die Anzeige und meldet das Verarbeitungsergebnis zurück. Jede Stufe besitzt damit einen definierten Eingang, einen Ausgang und eine eigene Fehlergrenze. Tabelle 3.4 fasst diese Zuordnung zusammen.

Tabelle 3.4: Stufen des Ende-zu-Ende-Datenpfads und ihre Verantwortungsgrenzen

Stufe	Baustein	Beitrag zum Gesamtsystem	Überwachung und zuständige Recovery-Ebene
1: GPS	weather-gps.service	NMEA-Daten des GPS-Empfängers → Positionszustand in gps.json	Fehlende oder veraltete Position kennzeichnen; Dienst isoliert neu starten
2: Wetterabruf	weather-mwft.service	Positionszustand; Netzwerkpfad als Transport; Wetterdienstantwort → Wetterzustand in weather.json	Abruffehler als degradierten Zustand weitergeben; Prototyp unterscheidet Netzwerk- und Dienstursache noch nicht; Prozessfehler durch isolierten Dienstneustart behandeln
3: UART	weather-uart.service und UART	Positions- und Wetterzustand → serieller Datenrahmen; PING/PONG als Rückmeldepfad	Fehlende Antwort über PING/PONG erkennen; Übertragung im nächsten Zyklus erneut versuchen; Dienst bei Prozessfehler isoliert neu starten
4: LCD	STM32-Firmware	Rahmen validieren → LCD aktualisieren; Verarbeitung mit OK, ERR oder PONG rückmelden	Fehlerhaften Rahmen verwerfen; LCD lokal reinitialisieren; bei kritischem Firmware-Stillstand IWDG-Reset

Die Überwachung bildet keine zusätzliche Stufe des Nutzdatenpfads, sondern liegt quer dazu. systemd überwacht die drei Linux-Dienste einzeln, der STM32-Supervisor überwacht die hardwarenahen Funktionen, und die Hardware-Watchdogs beider Plattformen bilden die jeweils letzte Eskalationsebene. Dadurch lässt sich jede Komponente sowohl ihrem Beitrag zur Nutzfunktion als auch der für sie zuständigen Recovery-Ebene zuordnen.

3.3.2 Physischer Aufbau und Plattformgrenzen

Der Hardwareaufbau setzt den in Abbildung 3.1 beschriebenen Datenpfad physisch um. Der GPS-Empfänger ist über USB mit dem Raspberry Pi verbunden. Dort erscheint er über die Geräteklasse USB-CDC als virtuelle serielle Schnittstelle /dev/ttyACM0 und

liefert NMEA-Datensätze mit 9.600 Bd; die Einheit Baud bezeichnet dabei die Symbolrate.

Für die Weitergabe an den STM32 wird auf dem Raspberry Pi /dev/serial0 und auf dem STM32 das serielle Hardwaremodul USART1 verwendet. PA9 dient dabei als Sendeleitung (TX), PA10 als Empfangsleitung (RX). Die Verbindung arbeitet mit 115.200 Bd im Format 8N1, also mit acht Datenbits, ohne Paritätsbit und mit einem Stoppbit.

Der STM32 steuert das LCD2004 über I2C1 an den Pins PB6 und PB7 an. SCL führt dabei den Takt und SDA die Daten. Die Einstellung Alternate Function ordnet die Pins der jeweiligen Peripheriefunktion anstelle der allgemeinen Ein-/Ausgabefunktion zu; die Open-Drain-Ausgänge verwenden die gemeinsamen Pull-up-Widerstände der Busleitungen. Der LCD-Adapter wurde ausschließlich mit der 7-Bit-Adresse 0x27 bei einer Busfrequenz von 100 kHz betrieben. Als Systemtakt dient der interne 16-MHz-Hochgeschwindigkeitsoszillator HSI16. Die Registerkonfiguration orientiert sich am STM32-Referenzhandbuch [4, S. 1151-1183, Abschn. 38.1-38.4.10]. Tabelle 3.5 fasst die Anschlüsse und Konfigurationswerte zusammen.

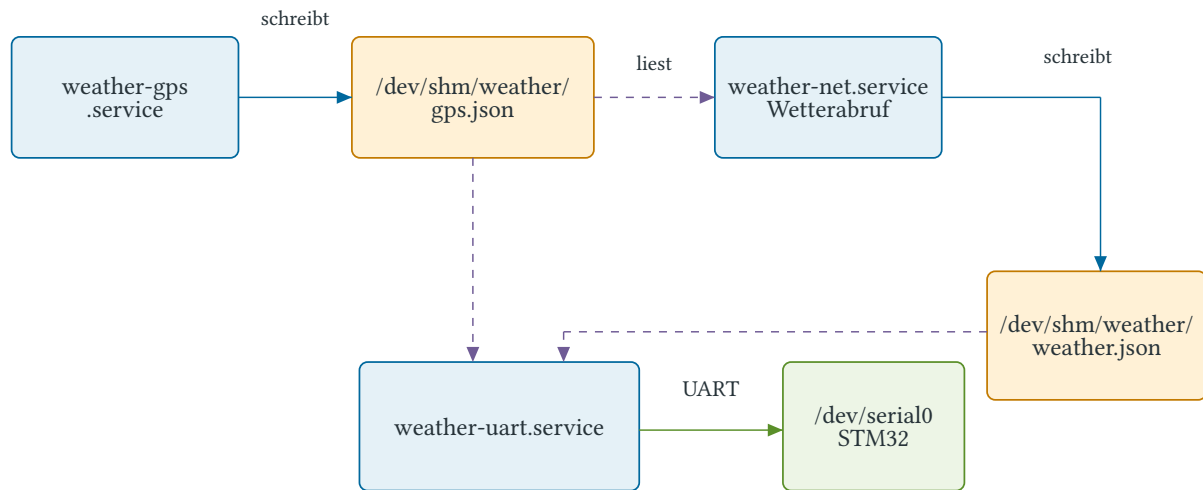
Tabelle 3.5: Wesentliche Schnittstellen und Konfigurationswerte

Funktion	Anschluss	Konfiguration
GPS am Raspberry Pi	/dev/ttyACM0	USB-CDC, 9.600 Bd, NMEA
UART am Raspberry Pi	/dev/serial0	115.200 Bd, 8N1
USART1 am STM32	PA9 (TX), PA10 (RX)	Alternate Function 7, 115.200 Bd
I2C1 am STM32	PB6 (SCL), PB7 (SDA)	Alternate Function 4, Open Drain, 100 kHz
LCD2004 mit I2C-Adapter	Adresse 0x27	20 Zeichen × 4 Zeilen, HD44780-kompatibler 4-Bit-Modus
Systemtakt STM32	HSI16	16 MHz

3.4 Raspberry-Pi-Teilsystem

Der Raspberry Pi realisiert die ersten drei Stufen aus Tabelle 3.4. Ausgangspunkt sind die NMEA-Daten des GPS-Empfängers. Aus dem daraus erzeugten Positionszustand wird die Anfrage an den Wetterdienst abgeleitet; dessen Antwort ergänzt die positionsbezogenen Wetterdaten. Ausgang des Teilsystems ist ein aus beiden Zuständen gebildeter UART-Rahmen.

weather-gps.service schreibt den Positionszustand nach /dev/shm/weather/gps.json. weather-net.service liest diesen Zustand und erzeugt daraus weather.json. weather-uart.service liest beide Dateien, bildet den seriellen Rahmen und besitzt exklusiv /dev/serial0. Die Zustandsdateien sind damit zugleich Übergabepunkte des Nutzdatenpfads und Beobachtungspunkte für Aktualität und Fehlerzustand. Abbildung 3.2 zeigt den gerichteten Datenfluss und die Grenzen der drei Prozesse.



Blau/durchgezogen: Schreiben · Violett/gestrichelt: Lesen · Grün: UART

Abbildung 3.2: Detailansicht des Raspberry-Pi-Teilsystems mit entkoppelten Diensten und Zustandsdateien

3.4.1 Gemeinsame Konfiguration und Zustandsaustausch

Zentrale Konstanten definieren Schnittstellen, Timeouts und Zyklen. Der GPS- und Wetterabrufdienst laufen mit einer Periode von 10 s, der UART-Dienst mit 5 s. Ein Zustandsdokument wird nach 30 s ohne Aktualisierung als veraltet behandelt. Die letzte Position darf höchstens zwei Stunden alt sein, bevor sie nicht mehr als Fallback verwendet wird. Der folgende Auszug zeigt die dafür maßgeblichen Konfigurationswerte:

```

GPS_PORT = "/dev/ttyACM0"
UART_PORT = "/dev/serial0"
BAUDRATE_GPS = 9600
BAUDRATE_UART = 115200

GPS_POLL_INTERVAL_SECONDS = 10
NET_POLL_INTERVAL_SECONDS = 10
UART_POLL_INTERVAL_SECONDS = 5
STATE_MAX_AGE_SECONDS = 30
MAX_POSITION_AGE_SECONDS = 2 * 60 * 60

USE_UART_CHECKSUM = True
STM32_RESPONSE_TIMEOUT_SECONDS = 2.0
STM32_SEND_RETRIES = 1

```

Die Dienste tauschen kleine JSON-Dokumente in `/dev/shm/weather` aus. Das Verzeichnis liegt auf `tmpfs`, wodurch periodische Schreibzugriffe auf die SD-Karte vermieden werden. Ein Zustand enthält neben Nutzdaten einen Zeitstempel. Die Schreibfunktion erzeugt zuerst eine temporäre Datei und ersetzt anschließend den Zielpfad atomar. Dadurch liest ein anderer Dienst entweder den alten oder den vollständig neuen Zustand, nicht jedoch ein teilweise geschriebenes JSON-Dokument.

Diese Entkopplung begrenzt zugleich die Recovery-Domäne: Ein betroffener Dienst kann neu gestartet werden, ohne die beiden anderen Prozesse ebenfalls zurückzusetzen. Der flüchtige Speicher verhindert außerdem veraltete Boot-Zustände, weil nach einem Neustart keine alten Dateien existieren, die fälschlich als aktuelle Messwerte interpretiert werden könnten.

3.4.2 GPS-Erfassung

Der GPS-Dienst bildet den Eingang des Nutzdatenpfads. Er öffnet `/dev/ttyACM0`, liest RMC-Sätze und akzeptiert nur Datensätze mit gültigem Status. Breiten- und Längengrad werden aus Grad und Minuten in Dezimalgrad umgerechnet. Ein gültiger Fix aktualisiert `fix_ts`, den Zeitstempel der letzten gültigen Position; ein fehlendes Gerät, ein Timeout oder ein ungültiger Status führt zu `have_fix=false`.

Der Zeitstempel ermöglicht den nachgelagerten Diensten, einen aktuellen Fix von einer noch zulässigen letzten Position zu unterscheiden. Der Prozess bleibt bei einem Eingabefehler aktiv und öffnet das Gerät im nächsten Zyklus erneut. Die lokale Recovery besteht somit zunächst im erneuten Gerätezugriff; erst ein Prozessfehler führt zum isolierten Dienstneustart.

3.4.3 Wetterabruf und Abgrenzung der Fehlerursachen

Der Wetterabrufdienst mit dem technischen Namen `weather-net.service` liest eine aktuelle oder noch zulässige letzte Position und ruft die Open-Meteo-Schnittstelle über HTTPS ab [17, Abschn. „Weather Forecast API“]. Sein Ausgang ist der Wetterzustand in `weather.json`, der zusammen mit der verwendeten Position die Eingabe des UART-Dienstes bildet.

Fachlich sind mindestens zwei Fehlerdomänen zu unterscheiden. Ein Fehler des Netzwerkpfads liegt vor, wenn die lokale Schnittstelle, Adressierung oder Route nicht verfügbar ist oder Namensauflösung, TCP-Verbindung beziehungsweise TLS-Aufbau scheitern. Nach einem begrenzten Retry ist hier zunächst die lokale Konnektivität zu diagnostizieren; nur bei einem bestätigten lokalen Fehler wäre eine Reinitialisierung der Netzwerkschnittstelle oder ihrer Konfiguration angemessen. GPS-Erfassung und UART-Übertragung bleiben davon unabhängig aktiv.

Ein Fehler des Wetterdienstes liegt dagegen vor, wenn der Netzwerkpfad grundsätzlich funktioniert, der Anbieter aber beispielsweise mit einem Dienstfehler oder einer Ratenbegrenzung antwortet. Auch eine syntaktisch oder semantisch nicht auswertbare Antwort bildet eine eigene Datenfehlerklasse. In diesen Fällen würde ein Zurücksetzen der Netzwerkschnittstelle die Ursache nicht beseitigen. Angemessen sind das Kennzeichnen nicht verfügbarer Wetterdaten, ein dienstspezifischer Retry mit Backoff und das Verwerfen ungültiger Antworten; ein Plattform-Reset ist wegen einer fortbestehenden externen Störung nicht vorgesehen.

Ein ursachenspezifisches Zustandsmodell benötigt deshalb zwei getrennte Größen: NET_PATH mit den Werten OK, ERR und UNKNOWN für die Konnektivität sowie WEATHER mit OK, SERVICE_ERR, DATA_ERR und UNKNOWN für Dienst und Antwort. Bei NET_PATH=ERR ist der Wetterdienst nicht beobachtbar; WEATHER muss daher UNKNOWN sein. SERVICE_ERR oder DATA_ERR sind nur bei nachgewiesenem NET_PATH=OK zulässig. Daraus folgen getrennte Reaktionen: Netzwerkdiagnose und gegebenenfalls Wiederverbindung bei NET_PATH=ERR, aber API-spezifischer Retry beziehungsweise Antwortverwerfung ohne Netzwerketat bei einem WEATHER-Fehler.

Der implementierte Prototyp besitzt diese gestufte Diagnose noch nicht. Er fasst Verbindungs-, Dienst- und Antwortfehler in weather.json als ok=false mit reason=net_error zusammen und behandelt sie mit demselben periodischen Retry. Derselbe Sammelfehler erscheint im Journal als INTERNET_ERROR und im UART-Protokoll als NET=ERR. Diese Bezeichnungen kennzeichnen somit ausschließlich einen nicht verfügbaren gültigen Wetterzustand und erlauben keine Aussage darüber, ob das Netzwerk selbst oder der Wetterdienst die Ursache ist. Die Weitergabe als degradiertes Zustand verhindert dennoch, dass GPS-Erfassung oder UART-Prozess unnötig zurückgesetzt werden.

3.4.4 Prozessüberwachung und Plattform-Recovery

Die Recovery des Raspberry Pi besitzt zwei Ebenen. Auf der Dienstebene überwacht systemd jeden der drei Prozesse separat. Auf der Plattformebene erkennt der Hardware-Watchdog einen Stillstand der gemeinsamen Linux-Ausführungsbasis. Damit folgt die Implementierung derselben Grenze wie der Nutzdatenpfad: Zunächst wird nur die fehlerhafte Komponente behandelt; erst wenn diese lokale Instanz nicht mehr handlungsfähig ist, wird die gesamte Plattform neu gestartet.

Dienstebene. Alle drei Dienste verwenden denselben Neustart- und Watchdog-Ansatz. Ein gemeinsames weather.target startet und stoppt die Prozesse als Gruppe, ohne ihre individuelle Steuerbarkeit aufzuheben. Der folgende Auszug zeigt die wesentlichen Einstellungen eines Dienstes:

```
[Service]
Type=notify
NotifyAccess=main
User=pi
WorkingDirectory=/home/pi/weather
ExecStart=/usr/bin/python3 -m rpi.service_gps
Restart=always
RestartSec=5
WatchdogSec=30s
```

Restart=always veranlasst nach jeder Prozessbeendigung einen Neustart; RestartSec=5 legt davor eine Wartezeit von fünf Sekunden fest. WatchdogSec=30s definiert die Frist für Fortschrittmeldungen. systemd stellt diesen Wert der Anwendung über

WATCHDOG_USEC in Mikrosekunden bereit. Type=notify verlangt zusätzlich, dass der Dienst seine abgeschlossene Initialisierung ausdrücklich meldet. Die Anwendung sendet das Lebenszeichen typischerweise nach der halben erlaubten Zeit und aktualisiert es auch während kontrollierter Wartezeiten, damit ein langer I/O-Timeout nicht fälschlich als Prozess-Hänger gilt. Die Semantik entspricht der offiziellen systemd-API [7, Abschn. WatchdogSec=, 8, Abschn. WATCHDOG=1].

Plattformebene. systemd aktualisiert zusätzlich periodisch /dev/watchdog0. Die Einstellung RuntimeWatchdogSec= ist in der systemd-Systemkonfiguration dokumentiert [20, Abschn. RuntimeWatchdogSec=]; die Kommunikation mit dem Gerät folgt der Linux-Watchdog-API [3, Abschn. „The simplest API“]. Der Versuch verwendet den BCM2835-Watchdog. Angefordert wurde RuntimeWatchdogSec=10s, der aktive Treiber meldete jedoch einen wirksamen Hardware-Timeout von 60 s. Das Werkzeug wdctl liest Konfiguration und Status des Geräts aus; der protokollierte Auszug ist in Quelltext C.6 wiedergegeben. Der Kernel-Panic-Test prüft die vollständige Kette aus ausbleibendem Keepalive, Hardware-Reset, Linux-Boot und automatischem Dienststart.

3.5 UART als Plattformgrenze

Die UART-Verbindung ist die einzige Schnittstelle zwischen den beiden internen Plattformen Raspberry Pi und STM32. Sie transportiert Nutzdatenrahmen zum STM32 sowie Bestätigungen, Fehlermeldungen und Heartbeat-Nachrichten in beiden Richtungen. Entwurf und Implementierung werden deshalb gemeinsam aus Sicht des Nachrichtenflusses, des Senders und des Empfängers beschrieben.

3.5.1 Nachrichten, Integrität und Betriebsmodi

Vor jedem Nutzdatenrahmen sendet der Raspberry Pi PING; der STM32 antwortet mit PONG. Dadurch wird nicht nur geprüft, ob die UART-Schnittstelle offen ist, sondern ob die Firmware eine Nachricht empfangen, verarbeiten und eine Antwort erzeugen kann. Ein Wetterdatenrahmen besitzt grundsätzlich folgende Form:

```
GPS=OK;NET=OK;TEMP=19.3;WIND=10.2;CODE=3;MODE=LIVE*XX
```

Die Nutzdaten werden durch Schlüssel-Wert-Paare beschrieben. GPS kennzeichnet die Verfügbarkeit eines aktuellen Positionsfix. Das historisch benannte Feld NET ist im implementierten Protokoll dagegen ein zusammengefasster Gültigkeitsstatus des Wetterdatenpfads: NET=ERR bedeutet, dass kein gültiger aktueller Wetterzustand vorliegt, nicht dass eine Störung der physischen oder allgemeinen Netzwerkkonnektivität nachgewiesen wurde. TEMP enthält die Lufttemperatur in Grad Celsius, WIND die Windgeschwindigkeit in Kilometern pro Stunde und CODE den von Open-Meteo gelieferten numerischen Wettercode nach der WMO-Systematik [17, Abschn. „Weather Forecast API“]. MODE unterscheidet aktuelle Positionsdaten (LIVE) von der gekennzeichneten Verwendung einer letzten noch zulässigen Position (LAST); zwei Bindestriche stehen für einen nicht verfügbaren Wert.

Nach dem Sternzeichen folgt mit XX eine zweistellig hexadezimal dargestellte XOR-Prüfsumme über alle vorherigen ASCII-Bytes. Der STM32 akzeptiert einen Rahmen erst nach erfolgreicher Prüfung und bestätigt ihn mit OK; diese Antwort wird im Folgenden auch als Daten-ACK bezeichnet. Ein ungültiger Rahmen wird mit ERR:CHECKSUM abgelehnt. Bei Überschreitung des Empfangspuffers wird ERR:BUFFER zurückgegeben.

MODE=LIVE kennzeichnet aktuelle Daten auf Grundlage eines gültigen GPS-Fix. Bei Verlust des GPS-Empfangs bleiben zuletzt bekannte Daten verfügbar und werden mit MODE=LAST sowie GPS=ERR gekennzeichnet. Dadurch bleibt der Systemzustand transparent; veraltete Daten werden nicht als aktuelle Daten dargestellt. Das Protokoll stellt damit sowohl Nutzdaten als auch die Signale für Übertragungs- und Verarbeitungsfehler bereit.

3.5.2 Sendeimplementierung auf dem Raspberry Pi

Nur weather-uart.service öffnet /dev/serial0. Nach erfolgreichem PING/PONG-Zyklus kombiniert der Dienst die Zustände der beiden anderen Prozesse. Bei einem gültigen aktuellen Fix überträgt er GPS=OK;MODE=LIVE. Fehlt ein aktueller Fix, ist gps.json aber jünger als 30 s und die letzte gültige Position höchstens zwei Stunden alt, wird GPS=ERR;MODE=LAST übertragen. Ohne eine solche zulässige Fallback-Position werden GPS=ERR und die übrigen Nutzfelder mit zwei Bindestrichen als nicht verfügbar gekennzeichnet. Ein aus beliebiger Ursache fehlgeschlagener Wetterabruf wird im derzeitigen Protokoll als NET=ERR dargestellt. Die XOR-Prüfsumme wird über die ASCII-Bytes vor dem Stern berechnet:

```
def checksum_xor(text: str) -> int:
    value = 0
    for byte in text.encode("ascii"):
        value ^= byte
    return value

frame = f"{payload}*{checksum_xor(payload):02X}\n"
```

Der Dienst durchsucht mehrere Antwortzeilen, weil der STM32 vor dem finalen PONG oder OK zusätzliche Diagnosemeldungen ausgeben kann. STM32_CONNECTION_ERROR bezeichnet im Raspberry-Pi-Journal einen fehlgeschlagenen PING/PONG-Zyklus; RPI_LINK_RECOVERED meldet auf STM32-Seite die anschließend wiederhergestellte Verbindung. Bleibt die finale Bestätigung aus, wird der Rahmen einmal wiederholt; anschließend wird der Fehler protokolliert, ohne den Prozess zu beenden.

3.5.3 Empfang, Validierung und Resynchronisation auf dem STM32

Die UART-Routine arbeitet nicht blockierend und besitzt eine feste Rahmenlänge. Bei Überschreitung wird der laufende Rahmen verworfen und ERR:BUFFER gesendet. Ein vollständiger Wetterrahmen wird erst nach Prüfung des Stern-Trennzeichens und der

XOR-Prüfsumme geparkt. Ein Fehler führt zu ERR:CHECKSUM; ein gültiger Rahmen zu OK.

Die Tests U1 und U2 prüfen zusätzlich, ob nach dem Fehler sofort wieder ein PING und ein gültiger Wetterrahmen verarbeitet werden können. Damit wird nicht nur die Erkennung, sondern auch die Wiederherstellung der Parser-Synchronisation bewertet. Nach erfolgreicher Validierung werden die Nutzdaten an den LCD-Treiber weitergegeben; die Antwort schließt zugleich den Rückmeldepfad zum Raspberry Pi.

3.6 STM32-Teilsystem

Der STM32 bildet die Ausgabe- und hardwarenahe Überwachungsebene. Sein Eingang sind die an der UART-Grenze validierten Daten und Heartbeat-Nachrichten. Seine sichtbare Nutzfunktion ist die LCD-Darstellung; parallel überwachen Bootdiagnose, Supervisor und IWDG den Fortschritt der Firmware sowie den Zustand der Peripherie.

3.6.1 Firmwarestruktur, Bootablauf und Diagnose

Die Firmware ist CMSIS-basiert und in Treiber-, Diagnose- und Anwendungsmodule zerlegt. Zu den Modulen gehören Systemzeit, UART, I2C, LCD2004, Nachrichtenparser, Supervisor, Power-Monitor, Bootdiagnose und IWDG. CMSIS stellt die standardisierten Cortex-M-Kern- und Registerdefinitionen bereit [21, Abschn. „CMSIS-Core“]; die Peripherieregister werden direkt konfiguriert.

Nach dem Reset werden Takt, SysTick, UART, I2C, Power-Monitor und Supervisor initialisiert. SysTick stellt dabei die periodische Zeitbasis der Firmware bereit. Die Bootdiagnose prüft I2C/LCD, UART-Kommunikation und den Watchdog. Zusätzlich werden die RCC-Resetflags gelesen und anschließend gelöscht; diese Statusbits des Reset-and-Clock-Control-Blocks dokumentieren die vorherige Reset-Ursache. Die Diagnose unterscheidet unter anderem Pin-, Software- und IWDG-Reset. Erst nach Abschluss wechselt der Supervisor vom Boot- in den Laufzeitmodus. Diesen Eintritt meldet die Firmware mit APP_RUNTIME; beim Test H1 dient die Diagnosemeldung als funktionaler Endzeitpunkt.

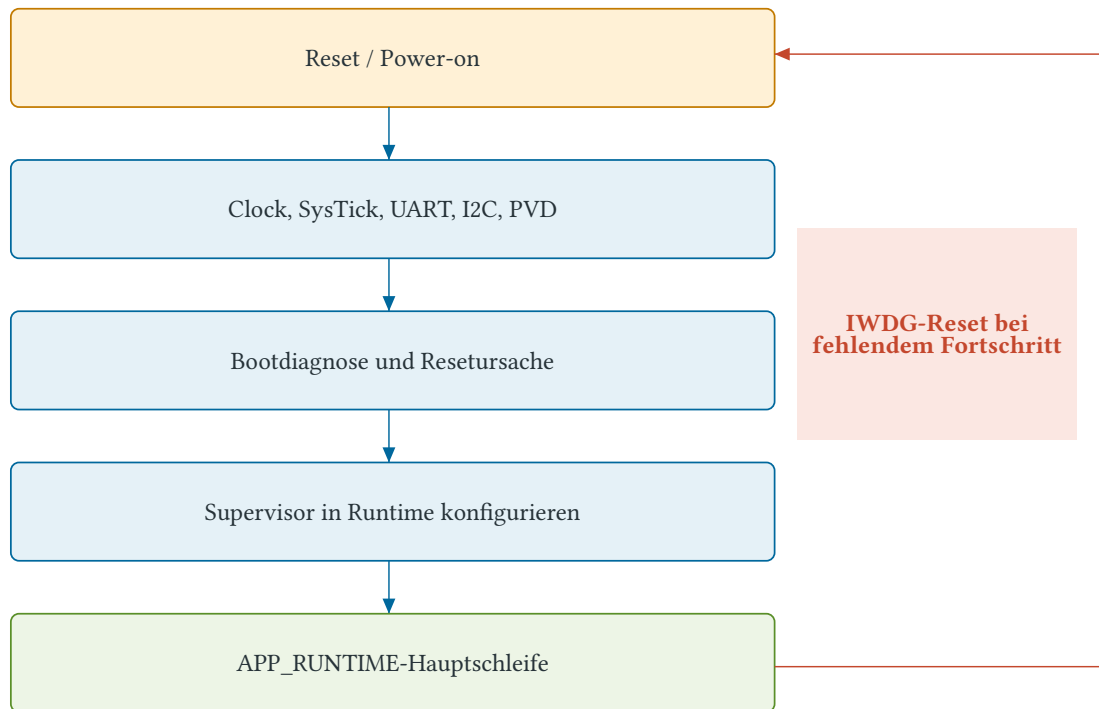


Abbildung 3.3: Vereinfachter Boot- und Recovery-Ablauf des STM32

3.6.2 LCD-Ausgabe und lokale I2C-Recovery

Nach einem gültigen UART-Rahmen aktualisiert der LCD-Treiber die Wetter- und Statusanzeige. Die LCD-Gesundheit wird jede Sekunde geprüft. Einzelne Fehlschläge ändern den Zustand noch nicht; erst drei aufeinander folgende Fehler bestätigen den Ausfall.

Der Treiber begrenzt I2C-Wartezeiten durch Software-Timeouts, damit ein dauerhaft gesetztes BUSY-Flag die Hauptschleife nicht unbegrenzt blockiert. Anschließend kann er die Peripherie zurücksetzen und bis zu neun SCL-Pulse zur Busfreigabe erzeugen. Bei bestätigtem Ausfall wird `LCD_DRIVER=FAIL` und der globale Modus `DEGRADED` gesetzt. Hauptschleife und UART-Verbindung bleiben dabei aktiv.

Nach drei erfolgreichen Proben wird das LCD neu initialisiert, der Recovery-Zähler erhöht und der Modus wieder zu `NORMAL` geändert. Die Konfiguration `RESET_AFTER_LCD_STABLE_RECOVERY=0` stellt sicher, dass die Wiederkehr des nicht kritischen Displays keinen vollständigen Mikrocontroller-Reset auslöst. Damit bleiben Erkennung und Recovery auf die betroffene Peripherie begrenzt.

3.6.3 Supervisor, Gesundheitsmodell und IWDG

Bootdiagnose, Hauptschleife, UART-Verarbeitung und LCD-Treiber melden ihren Zustand an den Supervisor. Die internen Bezeichner ordnen die überwachten Funktionen eindeutig zu: `BOOT` bezeichnet die Startdiagnose, `MAIN_LOOP` den Fortschritt der Hauptschleife, `UART_DRIVER` und `LCD_DRIVER` die beiden Peripherietreiber, `RPI_LINK` die Heartbeat-Verbindung zum Raspberry Pi und `POWER_SUPPLY` die Spannungsüberwachung. `BOOT` wird daneben auch als globaler Systemzustand wäh-

rend des Starts verwendet; aus dem Kontext ist jeweils ersichtlich, ob der Supervisor-Eintrag oder der globale Modus gemeint ist.

Jeder Eintrag enthält Aktivierung, Kritikalität, Gesundheitszustand und Zeitpunkt des letzten Fortschritts. Für kritische Subsysteme gelten Laufzeitgrenzen von typischerweise 3 s; RPI_LINK und LCD_DRIVER sind nicht kritisch für den IWDG-Refresh. Bei einem bestätigten nicht kritischen Fehler wechselt das System zu DEGRADED. Ein Stillstand der Hauptschleife verhindert dagegen die Freigabe des IWDG-Lebenszeichens. Diagnosebefehle liefern Alter, Zustand und Kritikalität der Subsysteme sowie kumulative Fehler- und Recovery-Zähler.

Der IWDG wird mit dem LSI-Oszillator betrieben. Im Normalbetrieb erfolgt ein Refresh nur, wenn `supervisor_all_critical_ok()` wahr ist. Damit ist die Watchdog-Aktualisierung an den Fortschritt der kritischen Funktionspfade und nicht nur an einen periodischen Interrupt gekoppelt. Der Diagnosebefehl HANG bestätigt zunächst mit HANGING, setzt eine verkürzte IWDG-Periode und verbleibt anschließend absichtlich in einer Endlosschleife. Nach dem Reset weist die Bootdiagnose den IWDG als Ursache aus. Dieser Testpfad ist ausschließlich für die Evaluation vorgesehen.

3.6.4 Spannungsüberwachung

Der STM32-PVD und ein Supervisor-Eintrag POWER_SUPPLY sind in der Firmware vorbereitet. Die Zustandsänderung wird entprellt und kann als kritisch bewertet werden. Ergänzend wurde das Verhalten der vollständigen Nucleo-Platine bei schrittweise abgesenkter VIN-Spannung untersucht. Die Implementierung bildet damit einen eigenen Beobachtungspfad; eine Sicherung des Ausführungszustands in nichtflüchtigem Speicher ist nicht Bestandteil des Systems.

3.7 Systemweites Recovery-Zusammenspiel

Im Normalbetrieb entsteht aus den empfangenen NMEA-Daten zunächst der Positionszustand in `gps.json`. Der Wetterabrufdienst ergänzt ihn zum Wetterzustand in `weather.json`, aus dem der UART-Dienst den seriellen Rahmen bildet. Der STM32 validiert diesen Rahmen, meldet das Verarbeitungsergebnis zurück und aktualisiert bei gültigen Daten das LCD. Zustandsdateien und UART-Verbindung sind damit sowohl Übergabe- als auch Beobachtungspunkte des Gesamtsystems.

Bei einem Fehler greift zunächst die Recovery-Ebene, die der betroffenen Grenze am nächsten liegt. Ungültige Rahmen werden verworfen, externe GPS- oder Wetterfehler als degradierte Zustand weitergegeben und fehlerhafte Linux-Dienste durch `systemd` isoliert neu gestartet. Auf dem STM32 werden Peripheriefehler lokal behandelt; nur ein kritischer Firmware-Stillstand führt über den IWDG zum Reset. Erst wenn die jeweilige Plattform ihre Recovery nicht mehr selbst ausführen kann, wird auf die systemweite Reset-Ebene eskaliert.

3.7.1 Eskalationshierarchie

Das Recovery-Konzept ordnet die Maßnahmen nach zunehmender Eingriffstiefe:

Ebene 1: Datenrahmen verwerfen: ungültige Prüfsumme oder zu langer Rahmen.

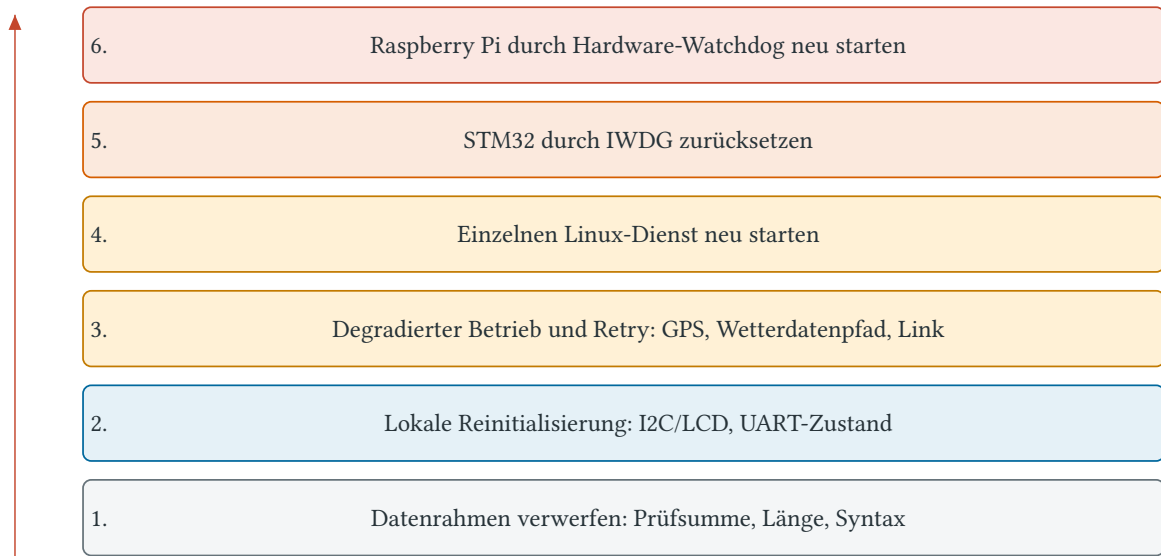
Ebene 2: Lokale Komponente reinitialisieren: LCD-/I2C-Fehler.

Ebene 3: Degradierter Betrieb und Retry: GPS-Fehler oder nicht verfügbarer Wetterdatenpfad.

Ebene 4: Einzelnen Dienst neu starten: Prozessabsturz oder Service-Watchdog-Time-out.

Ebene 5: Mikrocontroller zurücksetzen: Firmware-Hänger des STM32.

Ebene 6: Raspberry Pi neu starten: Kernel-Hänger oder vollständiger Systemfehler.



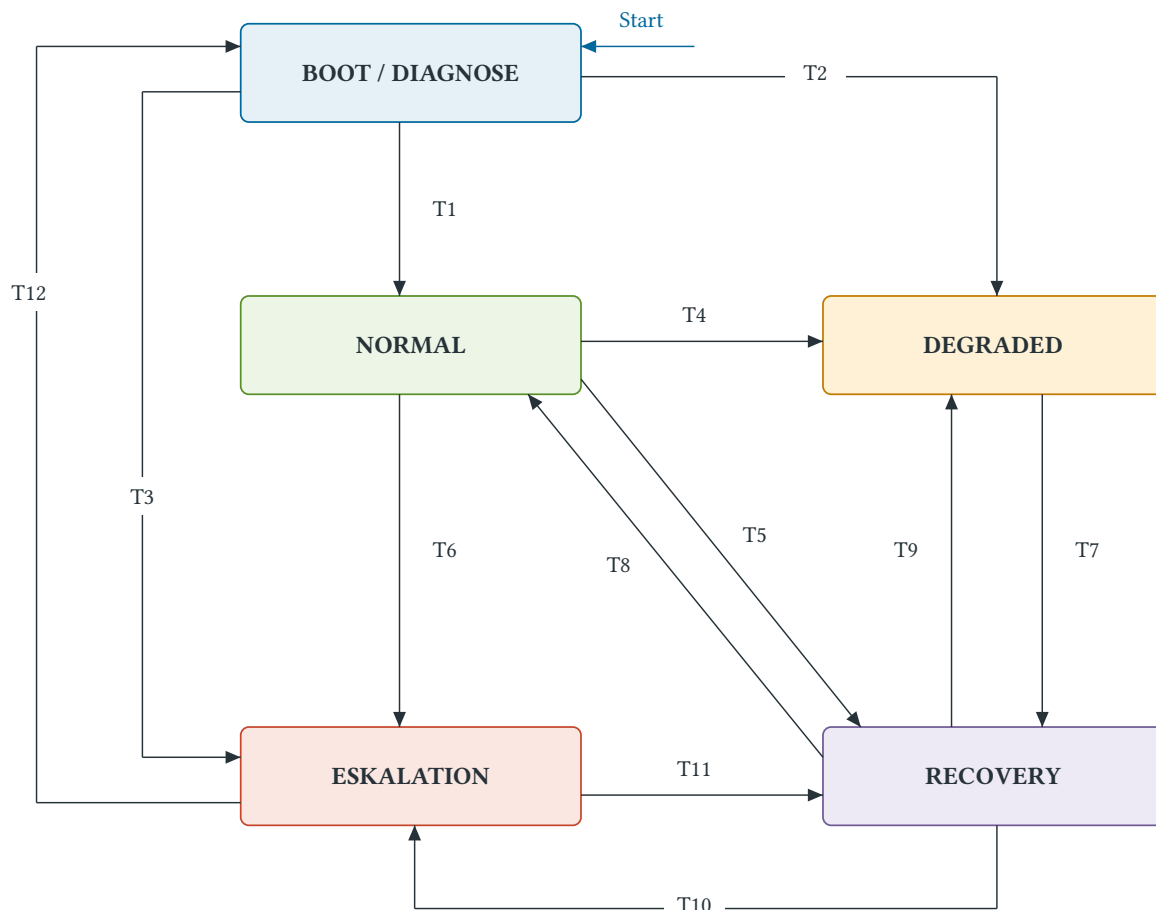
Steigende Eingriffstiefe von unten nach oben

Abbildung 3.4: Eskalationshierarchie der Recovery-Strategien

Die Ebenen konkretisieren die in Tabelle 3.4 ausgewiesenen Recovery-Grenzen: Die Ebenen 1 und 2 behandeln Daten- und Peripheriefehler unmittelbar auf dem STM32. Die Ebenen 3 und 4 betreffen externe Ressourcen beziehungsweise einzelne Raspberry-Pi-Dienste. Die Ebenen 5 und 6 setzen schließlich die jeweilige Ausführungsbasis zurück. Diese Reihenfolge minimiert die Eingriffstiefe und erhält funktionsfähige Zustände so lange wie möglich.

3.7.2 Verteilter Recovery-Zustandsautomat

Der globale Systemzustand unterscheidet zwischen stabilen Betriebszuständen und transienten Recovery-Zuständen. Der in Abbildung 3.5 dargestellte Automat ist eine systemweite Abstraktion der verteilten Implementierung. Er ist nicht als zusätzliche monolithische Softwarekomponente zu verstehen: Auf dem STM32 werden BOOT, NORMAL und DEGRADED durch Bootdiagnose und Supervisor abgebildet; auf dem Raspberry Pi setzen die systemd-Zustände, Restart-Policies und Service-Watchdogs die Übergänge für einzelne Prozesse um. Die Zustände RECOVERY und ESKALATION fassen die jeweils ausgeführten Maßnahmen über beide Plattformen hinweg zusammen.



Abrupter Spannungsverlust beendet die Softwareausführung unabhängig vom Zustand. Nach stabiler Spannungsrückkehr beginnt der Ablauf erneut in BOOT.

Abbildung 3.5: Systemweiter Zustandsautomat für Fehlerbehandlung und Recovery

Im Zustand NORMAL sind die kritischen Subsysteme gesund und die reguläre Nutzfunktion ist verfügbar. Ein bestätigter Ausfall einer nicht kritischen oder externen Komponente führt zu DEGRADED; dort bleiben noch funktionsfähige Pfade aktiv und der Fehler wird nach außen sichtbar gemacht. RECOVERY ist ein transienter Zustand. Je nach Fehlerdomäne umfasst er das Verwerfen eines fehlerhaften Rahmens, einen begrenzten Retry, das erneute Öffnen eines Geräts, die I2C-Busfreigabe mit anschließender LCD-Initialisierung oder die Verifikation nach einem Dienstneustart. Der Übergang zurück zu NORMAL ist nur nach einem positiven Funktionsnachweis zulässig. Bleibt eine nicht kritische Störung bestehen, führt der Automat zurück nach DEGRADED.

ESKALATION wird erreicht, wenn ein kritisches Fortschrittskriterium verletzt wird oder die lokale Recovery innerhalb ihrer festgelegten Grenze erfolglos bleibt. Ein einzelner Linux-Dienst wird zunächst isoliert neu gestartet und anschließend im Zustand RECOVERY funktional geprüft. Kann die betroffene Ausführungsbasis selbst keine Recovery mehr durchführen, lösen IWDG beziehungsweise Raspberry-Pi-Hardware-Watchdog einen Reset aus; der nächste ausführbare Zustand ist dann BOOT. Damit

bildet der Automat das Prinzip der feingranularen Recovery mit kontrollierter Eskalation ab [2, S. 5 und 8; 6, S. 31–32; 11, S. 16].

Tabelle 3.6: Übergänge des systemweiten Recovery-Zustandsautomaten

ID	Übergang	Auslöser beziehungsweise Bedingung
T1	BOOT → NORMAL	Bootdiagnose erfolgreich; alle benötigten Funktionen verfügbar.
T2	BOOT → DEGRADED	Nicht kritische Ressource fehlt; Nutzfunktion bleibt möglich.
T3	BOOT → ESKALATION	Kritische Initialisierung oder Fortschrittsnachweis fehlgeschlagen.
T4	NORMAL → DEGRADED	Bestätigter nicht kritischer LCD-, GPS-, Netzwerkpfad-, Wetterdienst- oder RPI-Link-Fehler.
T5	NORMAL → RECOVERY	Fehler ist in der betroffenen Domäne lokal behandelbar.
T6	NORMAL → ESKALATION	Kritisches Lebenszeichen oder Fortschritt der Hauptfunktion fehlt.
T7	DEGRADED → RECOVERY	Ressource wieder verfügbar oder periodischer Retry fällig.
T8	RECOVERY → NORMAL	Recovery verifiziert; alle benötigten Pfade wieder gesund.
T9	RECOVERY → DEGRADED	Teilfunktion bleibt nicht kritisch gestört.
T10	RECOVERY → ESKALATION	Interne lokale Recovery fehlgeschlagen oder kritischer Fehler fortbestehend; ein rein externer Fehler bleibt DEGRADED.
T11	ESKALATION → RECOVERY	Dienstneustart abgeschlossen; Nutzfunktion wird geprüft.
T12	ESKALATION → BOOT	IWDG-Reset des STM32 oder Hardware-Reboot des Raspberry Pi ausgelöst.

3.7.3 Diagnose und Nachvollziehbarkeit

Die Linux-Dienste protokollieren Ereignisse im systemd-Journal und Status in JSON-Dateien; der STM32 liefert Diagnosen über UART. Vor jeder Fehlerinjektion sichern die Testskripte einen Journal-Cursor, Konsolenausgabe, Rohlogs, CSV-Einzelwerte und statistische Zusammenfassungen. Das vollständige Evaluationsverzeichnis wurde archiviert und mit SHA-256 versehen. Damit werden die betrieblichen Beobachtungspunkte zu reproduzierbaren Messbelegen.

3.8 Erfolgskriterien

Das integrierte Systemkonzept gilt als erfolgreich umgesetzt, wenn Fehler der vorgesehenen Klasse zugeordnet werden, ohne funktionsfähige Domänen zu beeinträchtigen; wenn die Recovery auf der kleinsten ausreichenden Ebene bleibt und auch hängende Prozesse erkennt; und wenn nach Wegfall der Störung der funktionale Endzustand ohne manuellen Neustart erreicht wird sowie Reset- und Neustartursachen nachvollziehbar bleiben.

4 Evaluation

Dieses Kapitel untersucht, ob die im Rahmen der Arbeit implementierten Watchdog- und Recovery-Strategien unterschiedliche Fehlerzustände zuverlässig erkennen, eingrenzen und automatisch beheben. Betrachtet werden Fehler auf mehreren Ebenen: fehlerhafte Kommunikationsdaten, Ausfälle von Peripherie und externen Verbindungen, abgestürzte oder hängende Linux-Dienste, ein Stillstand der Mikrocontroller-Firmware, ein Kernel-Hänger, ein vollständiger Ausfall der Spannungsversorgung sowie kontrolliert abgesenkte Versorgungsspannungen. Das Ziel der Evaluation besteht nicht ausschließlich im Nachweis einer Fehlererkennung. Ein Test gilt erst dann als erfolgreich, wenn der zuvor definierte funktionale Endzustand wieder erreicht wurde. Dazu zählen je nach Testfall beispielsweise ein gültiger GPS-Fix, eine erfolgreiche Wetterabfrage, ein bestätigter UART-Datenrahmen, der Supervisor-Modus NORMAL oder ein automatisch wiederhergestelltes Gesamtsystem.

4.1 Zielsetzung und Evaluationsfragen

Die Evaluation beantwortet insbesondere folgende Fragen:

1. Werden die vorgesehenen Fehlerzustände zuverlässig erkannt?
2. Wird der Fehlerzustand korrekt an nachgelagerte Komponenten weitergegeben?
3. Wird die jeweils kleinstmögliche geeignete Recovery-Ebene verwendet?
4. Kehrt das System ohne manuellen Eingriff in einen funktionsfähigen Zustand zurück?
5. Wie lange dauern Fehlererkennung, Fehlerweitergabe und funktionale Wiederherstellung?
6. Wie reproduzierbar sind die gemessenen Zeiten über mehrere Versuchsdurchläufe?

4.2 Evaluationsaufbau und Beobachtungspunkte

Die Systemarchitektur, ihre Schnittstellen und die Recovery-Hierarchie wurden in Kapitel 3 gemeinsam mit der Implementierung beschrieben. Für die Evaluation werden deshalb nur die konkrete Geräteauswahl, zusätzliche Messmittel und diejenigen Zustände ergänzt, an denen Fehlererkennung und funktionale Wiederherstellung beobachtet werden.

4.2.1 Hardware und externe Instrumentierung

Der Versuchsaufbau besteht aus einem Raspberry Pi 4 Model B, einem STM32-Nucleo-L432KC-Entwicklungsboard, einem unter der Vertriebsbezeichnung Binghe VK-162 erworbenen USB-GPS-Empfänger sowie einem LCD2004 mit AZDelivery-I2C-Adapter. Die intern erkannte GPS-Empfängereinheit meldete sich in den Versuchslogs als ublox-7- beziehungsweise UBX-G70xx-Hardware; die zugehörigen USB- und Startmeldungen sind in Anhang A dokumentiert. Aus den GPRMC-Datensätzen werden

Breiten- und Längengrad bestimmt; GPRMC bezeichnet einen RMC-Satz mit dem NMEA-Talker-Identifizier GP.

Der LCD-Adapter verwendet einen PCF8574-I/O-Expander [19, S. 4]; dessen technische Eigenschaften sind im NXP-Datenblatt dokumentiert [15, S. 1 und 5–8]. Für den ergänzenden Unterspannungstest wurde die NUCLEO-L432KC-Platine über VIN und GND von einem Labornetzteil versorgt und die eingestellte Spannung in 1-V-Schritten von 8 V bis 2 V abgesenkt. Beim Raspberry Pi wurde bei einer Sollspannung von 4,50 V zusätzlich `vcgencmd get_throttled` abgefragt. Das Kommando liefert aktuelle und seit dem Start gespeicherte Unterspannungs- und Drosselungszustände als Bitmaske.

Tabelle 4.1: Hardwarekomponenten des Versuchsaufbaus

Komponente	Funktion
Raspberry Pi 4 Model B	Positionsverarbeitung, Wetterabfrage, Dienstverwaltung, Fehlerbehandlung und Kommunikation mit dem STM32
STM32 Nucleo L432KC	Displaysteuerung, Kommunikationsvalidierung, Bootdiagnose, Supervisor und hardwarebasierte Watchdog-Recovery
Binghe VK-162 USB-GPS-Empfänger	Bestimmung der geografischen Position über NMEA-Datensätze
LCD2004 mit AZDelivery-I2C-Adapter (PCF8574)	Anzeige von Wetterdaten und aktuellem Systemzustand
UART-Verbindung	Bidirektionale Kommunikation zwischen Raspberry Pi und STM32
I2C-Verbindung	Kommunikation zwischen STM32 und LCD-Modul

4.2.2 Softwareseitige Beobachtungspunkte

Die Evaluation greift auf dieselben Zustände zurück, die im Betrieb die Recovery steuern. Auf dem Raspberry Pi zeigen `gps.json` mit `have_fix` und `fix_ts` sowie `weather.json` mit `ok` und `reason` die Verfügbarkeit und Aktualität der beiden Eingangszustände. Der Status in `weather.json` beschreibt jedoch nur den Erfolg des Wetterabrufs; ein unabhängiger Zustand der allgemeinen Netzwerkkonnektivität wird im Prototyp nicht erfasst. Das `systemd`-Journal dokumentiert Prozessstarts, Watchdog-Timeouts und Dienstneustarts. An der Plattformgrenze belegen PONG, OK und die Fehlermeldungen des UART-Protokolls, ob Nachrichten empfangen, validiert und beantwortet wurden. Auf dem STM32 markieren `APP_RUNTIME`, die Supervisor-Zustände und die Reset-Ursache den Übergang vom Boot zur funktionsfähigen Anwendung. `LCD_DRIVER`, `RPI_LINK` und die kumulativen Fehler- und Recovery-Zähler grenzen lokale Peripheriefehler von Kommunikations- und Firmwarefehlern ab. Die in Abschnitt 3.7 definierte Eskalationshierarchie legt für jeden Testfall die erwartete Reaktion fest; der Testkatalog in Abschnitt 4.4 ordnet diesen Reaktionen die konkrete Fehlerinjektion und das jeweilige Erfolgskriterium zu.

4.3 Messmethodik

4.3.1 Messgrößen

Die gezielte Fehlerinjektion ist ein etabliertes Mittel zur experimentellen Bewertung von Fehlererkennungs- und Recovery-Mechanismen [22, S. 75–76]. Für jeden Testfall wurden mindestens zwei Zeitpunkte betrachtet: der Beginn der Fehlerinjektion und das Erreichen eines definierten funktionalen Endzustands. Abhängig vom Testfall wurden zusätzlich Zwischenzeitpunkte protokolliert: Fehlerinjektionszeitpunkt: Zeitpunkt, zu dem der Fehler ausgelöst oder durch den Bediener bestätigt wurde. Fehlererkennungszeitpunkt: Zeitpunkt, zu dem der zuständige Dienst oder Supervisor den Fehler registrierte. Fehlerweitergabezeitpunkt: Zeitpunkt, zu dem der Fehlerstatus an eine nachgelagerte Komponente übertragen wurde. Recovery-Zeitpunkt: Zeitpunkt, zu dem die zuvor ausgefallene Teilfunktion erneut verfügbar war. Funktionale Wiederherstellung: Zeitpunkt, zu dem der vollständige Datenpfad wieder funktionierte und das Ergebnis bestätigt wurde. Die allgemeine End-to-End-Zeit ergibt sich aus

$$t_{\text{E2E}} = t_{\text{funktional}} - t_{\text{Fehler}} \quad (4.1)$$

Dabei bezeichnet t_{Fehler} den für den jeweiligen Testfall festgelegten Startzeitpunkt. Dieser ist je nach Versuch die automatische Fehlerinjektion, eine Bedienerbestätigung oder das erste externe DOWN-Ereignis. $t_{\text{funktional}}$ ist der Zeitstempel, an dem der vorab definierte funktionale Endzustand bestätigt wurde. Die konkreten Start- und Endereignisse werden je Testfall angegeben.

4.3.2 Monotone Zeitquelle und Journal-Cursor

Für Zeitdifferenzen innerhalb eines laufenden Linux-Systems wurde überwiegend `time.monotonic()` verwendet [23, Abschn. `time.monotonic()`]. Diese Zeitquelle ist gegen nachträgliche Änderungen der Systemzeit unempfindlich. Vor jeder Fehlerinjektion wurde außerdem ein `systemd-Journal-Cursor` gespeichert. Für die Auswertung wurden ausschließlich Logeinträge nach diesem Cursor berücksichtigt. Dadurch konnten ältere Meldungen nicht fälschlich einem aktuellen Versuch zugeordnet werden. Zu den ausgewerteten Meldungen gehörten unter anderem:

```
INTERNET_ERROR
STM32_CONNECTION_ERROR
STM32: PONG
STM32: OK
Watchdog timeout
Failed with result 'watchdog'
RPI_LINK_RECOVERED
```

Die implementierte Meldung `INTERNET_ERROR` ist dabei ein Sammelbezeichner für einen fehlgeschlagenen Wetterabruf. Sie identifiziert weder einen allgemeinen Netzerkausfall noch einen Fehler des Wetterdienstes eindeutig.

4.3.3 Externe Messung bei vollständigen Neustarts

Bei einem vollständigen Neustart kann ein lokal laufendes Messprogramm die Messung nicht fortsetzen. Deshalb wurde sowohl für P1, den vollständigen Spannungsverlust des Gesamtsystems, als auch für P2, die gezielt ausgelöste Kernel-Panic des Raspberry Pi, ein externer Rechner verwendet. Dieser prüfte die Erreichbarkeit des Raspberry Pi in Intervallen von 0,25 s per ICMP-Ping und protokollierte die Zustandswechsel UP, DOWN und erneut UP. Die externe Nichtverfügbarkeit wurde berechnet als

$$t_{\text{extern}} = t_{\text{UP}} - t_{\text{DOWN}} \quad (4.2)$$

Dabei ist t_{DOWN} der Zeitstempel des ersten als nicht erreichbar protokollierten Zustands und t_{UP} der Zeitstempel der erneut bestätigten Erreichbarkeit. Diese Zeit enthält je nach Testfall die verbleibende Watchdog-Zeit, den Reset, den Bootvorgang und die Wiederherstellung der Netzwerkschnittstelle.

4.3.4 Manuell ausgelöste Hardwarefehler

Die drei manuell injizierten Hardwarefehler werden bereits an dieser Stelle eingeführt: G1 bezeichnet das Entfernen und erneute Anschließen des USB-GPS-Empfängers. Bei C1 wird die UART-Sendeleitung vom Raspberry Pi zum STM32 getrennt und wiederverbunden. L1 bezeichnet die Trennung und Wiederverbindung der SDA-Leitung im LCD-/I2C-Pfad. Der vollständige Testkatalog mit erwartetem Verhalten und Erfolgskriterien folgt in Abschnitt 4.4. Da die Eingriffe manuell erfolgten, beginnt die Zeitmessung teilweise mit einer Bedienerbestätigung und nicht mit dem exakten elektrischen Umschaltzeitpunkt.

4.3.5 Wiederholungen und statistische Kenngrößen

Die risikoarmen und reproduzierbaren Tests wurden jeweils fünfmal durchgeführt. Der vollständige Spannungsverlust wurde aufgrund der zusätzlichen Belastung des Dateisystems dreimal getestet. Der Kernel-Hänger wurde zweimal ausgeführt; eine gültige externe Zeitmessung liegt für den zweiten Durchlauf vor. Für die wiederholten Tests wurden Erfolgsrate, Mittelwert, Median, Minimum, Maximum und Stichproben-Standardabweichung bestimmt. Die Erfolgsrate lautet

$$p_{\text{Erfolg}} = \frac{n_{\text{erfolgreich}}}{n_{\text{gesamt}}} \cdot 100\% \quad (4.3)$$

$n_{\text{erfolgreich}}$ ist die Anzahl der Durchläufe, die das festgelegte Erfolgskriterium erfüllen; n_{gesamt} bezeichnet alle ausgeführten Durchläufe des betreffenden Testfalls.

4.3.6 Dokumentation und Datenintegrität

Für jeden Durchlauf wurden Logdateien, Zustandsdateien und CSV-Messwerte in einem separaten Verzeichnis gespeichert. Nach Abschluss der Evaluation wurde das vollständige Verzeichnis archiviert. Der SHA-256-Prüfwert identifiziert genau den unveränderten Inhalt dieser Archivdatei; er ersetzt jedoch nicht deren Bereitstellung. Der Prüfwert des finalen Archivs lautet:

```
2e5134c401f3308dd6acbd33e57b0e516af9fa5dfe1a8a47ef6e2a3b633b5bc4
```

4.4 Definition der Testfälle

Die Testfallkennungen dienen als stabile Referenzen; ihr Buchstabe gruppiert die untersuchte Domäne: N bezeichnet hier den externen HTTPS-Wetterdatenpfad, G das GPS, C die Verbindung zwischen Raspberry Pi und STM32, H den Firmware-Hänger, S Linux-Dienste, U das UART-Protokoll, L den LCD-/I2C-Pfad und P systemweite Versorgungs- oder Plattformtests. Die Kennung N legt keine Ursache innerhalb des Wetterdatenpfads fest. Die Ziffer ist lediglich eine laufende Kennung innerhalb des Testkatalogs und beschreibt weder Schweregrad noch Ausführungsreihenfolge.

Für die Fehlerinjektion bezeichnet nftables den Linux-Paketfilter, mit dem ausgehende HTTPS-Verbindungen blockiert werden. Das Signal SIGSTOP pausiert einen Prozess, ohne ihn zu beenden, und modelliert hier einen Hänger; SIGKILL beendet ihn unmittelbar und modelliert einen Absturz. Magic SysRq ist eine Kernel-Schnittstelle, über die im Test gezielt eine Kernel-Panic ausgelöst wird. Ein Power-Cycle ist das vollständige Trennen und anschließende Wiederherstellen der Versorgung.

Tabelle 4.2: Übersicht der untersuchten Testfälle

ID	Fehlerzustand	Fehlerinjektion	Erwartetes Verhalten	Erfolgskriterium
N2	Blockierter HTTPS-Wetterdatenpfad	Blockieren ausgehender TCP-Verbindungen zu Port 443 mit nftables	Sammelstatus INTERNET_ERROR und NET=ERR; automatische Rückkehr zu NET=OK	Fehlerweitergabe und Recovery ohne Dienstneustart; keine Ursachendifferenzierung
G1	Verlust des GPS-Empfängers	Entfernen und erneutes Einstecken des USB-GPS	have_fix=false, Fallback MODE=LAST, anschließend MODE=LIVE	Automatische funktionale Wiederherstellung ohne Dienstneustart
C1	Unterbrechung Raspberry Pi–STM32	Trennen der TX-Leitung des Raspberry Pi zum RX-Eingang des STM32	Verbindungsfehler, danach erneut PONG und Daten-ACK	UART-Datenpfad ohne manuellen Prozessneustart wieder funktionsfähig
H1	Hänger der STM32-Firmware	Senden des Befehls HANG	IWDG-Reset, Bootdiagnose und Rückkehr in APP_RUNTIME	Reset-Ursache IWDG und erneutes PING/PONG bestätigt
S1	Hänger des GPS-Dienstes	Senden von SIGSTOP an den Prozess	systemd-Watchdog-Timeout, Prozessabbruch und Neustart	Neuer PID, Ergebnis watchdog und neuer GPS-Fix
S2	Absturz des GPS-Dienstes	Senden von SIGKILL an den Prozess	Sofortige Fehlererkennung und Neustart nach RestartSec	Neuer PID, Signalfehler und neuer GPS-Fix

ID	Fehlerzustand	Fehlerinjektion	Erwartetes Verhalten	Erfolgskriterium
S3	Hänger des UART-Dienstes	Senden von SIGSTOP an den Prozess	systemd-Watchdog-Neustart, PONG, Daten-ACK und RPI_LINK_RECOVERED	Vollständiger UART-Datenpfad wiederhergestellt
U1	Ungültige UART-Prüfsumme	Senden eines Datenrahmens mit absichtlich falscher XOR-Prüfsumme	Antwort ERR:CHECKSUM, danach Annahme eines gültigen Rahmens	Fehlerhafte Nachricht verworfen, UART bleibt funktionsfähig
U2	UART-Pufferlimit	Senden einer Nachricht mit 140 Zeichen	Antwort ERR:BUFFER, danach PONG und Annahme eines gültigen Rahmens	Keine dauerhafte Desynchronisation oder Blockade
L1	Ausfall der LCD-/I2C-Verbindung	Trennen und Wiederverbinden der SDA-Leitung	LCD_DRIVER=FAIL, Modus DEGRADED, danach lokale Recovery	Rückkehr zu LCD_DRIVER=OK und NORMAL ohne Reset
P1	Vollständiger Spannungsverlust	Abruptes Trennen und Wiederherstellen der Versorgung	Automatischer Boot beider Systeme und Start aller Dienste	GPS, Wetter und STM32-Kommunikation ohne manuellen Eingriff verfügbar
P2	Linux-Kernel-Hänger	Kernel-Panic über Magic SysRq bei kernel.panic=0	Neustart durch BCM2835-Hardware-Watchdog	Geänderte Boot-ID und vollständige Systemfunktion ohne Power-Cycle

Hinweis zu N2: Der Test blockiert gezielt ausgehende TCP-Verbindungen zu Port 443 und damit den HTTPS-Pfad der Wetterabfrage. Er prüft die Reaktion auf einen nicht verfügbaren Wetterdatenpfad. Der Versuch trennt weder einen lokalen beziehungsweise allgemeinen Netzwerkfehler von einem Fehler des Wetterdienstes noch untersucht er HTTP-Fehler oder ungültige API-Antworten. Entsprechend darf N2 nicht als Nachweis einer ursachenspezifischen Netzwerkd Diagnose interpretiert werden.

Ergänzend zum ursprünglichen Katalog wurde der Testfall P3 aufgenommen. P3a charakterisiert das Unterspannungsverhalten der NUCLEO-L432KC-Platine mit angeschlossenem LCD bei schrittweise abgesenkter VIN-Spannung; der qualitative Ablauf wurde fünfmal wiederholt. P3b dokumentiert beim Raspberry Pi den Statuswert von `vcgencmd get_throttled` bei einer am Labornetzteil eingestellten Spannung von 4,50 V.

4.5 Messergebnisse

4.5.1 Gesamtübersicht

Alle wiederholten funktionalen Recovery-Testfälle erreichten eine Erfolgsrate von 100 %. Tabelle 4.3 fasst die wichtigsten End-to-End-Messgrößen dieser Testfälle zusammen. Bei U1, U2 und L1 liegen die Zeiten im Millisekundenbereich. Die manuell ausgelösten Tests G1, C1 und L1 besitzen die in Abschnitt 4.3 beschriebenen Einschränkungen. Der

ergänzende Unterspannungstest P3 wird separat ausgewertet, da er ein qualitatives Platinenverhalten und keine End-to-End-Recovery-Zeit beschreibt.

Tabelle 4.3: Zusammenfassung der zentralen Messergebnisse

ID	Szenario und Hauptmessgröße	Läufe	Erfolg	Mittel [s]	Min–Max [s]
N2-F	HTTPS-Wetterdatenpfad blockiert Fehlerweitergabe bis NET=ERR	5	100 %	7,36	5,10–8,07
N2-R	HTTPS-Wetterdatenpfad freigegeben Funktionale Recovery bis NET=OK	5	100 %	10,10	9,95–10,32
G1-F	USB-GPS getrennt GPS-Fehlerweitergabe bis MODE=LAST	5	100 %	3,12	0,04–10,85
G1-R	USB-GPS verbunden Recovery bis MODE=LIVE	5	100 %	6,42	0,04–12,56
C1-F	Pi–STM32-Verbindung getrennt Bedienerbestätigung bis Fehler	5	100 %	3,80	2,26–5,02
C1-R	UART-Verbindung wiederhergestellt Funktionale Recovery bis ACK	5	100 %	4,73	0,13–13,30
H1	STM32-Firmware-Hänger HANG bis APP_RUNTIME	5	100 %	5,70	5,70–5,70
S1	GPS-Dienst hängt Funktionale Recovery	5	100 %	28,46	27,51–31,62
S2	GPS-Dienst stürzt ab Funktionale Recovery	5	100 %	5,76	5,62–6,10
S3	UART-Dienst hängt Funktionale Recovery	5	100 %	33,34	33,07–33,91
U1	Ungültige XOR-Prüfsumme Ablehnung des ungültigen Rahmens	5	100 %	0,010	0,010–0,011
U2	UART-Pufferüberlauf Erkennung des Überlaufs	5	100 %	0,018	0,018–0,019
L1-F	LCD-SDA getrennt Bestätigung bis LCD_DRIVER=FAIL	5	100 %	0,046	0,042–0,048
L1-R	LCD-SDA verbunden Bestätigung bis LCD_DRIVER=OK	5	100 %	0,044	0,042–0,045
P1	Vollständiger Spannungsverlust Externe Netzwerknichtverfügbarkeit	3	100 %	32,33	32,00–33,00
P2	Linux-Kernel-Hänger Einzelmessung der externen Netzwerknichtverfügbarkeit: 35,00 s (Durchlauf 02)	2	100 %	–	–

Für P2 wurden zwei funktional erfolgreiche Durchläufe ausgeführt. Eine verwertbare externe Zeitmessung liegt jedoch nur für Durchlauf 02 vor. Der Wert von 35 s ist deshalb ein Einzelwert und kein Mittelwert; Streuungsmaße werden für P2 nicht angegeben.

4.5.2 N2: Blockierung und Wiederherstellung des HTTPS-Wetterdatenpfads

Zum Auslösen der Störung blockierte eine nftables-Regel ausgehende TCP-Verbindungen zu Port 443. Jeder der fünf Durchläufe führte erwartungsgemäß zum protokollierten Sammelzustand NET=ERR; weder der Wetterabruf- noch der UART-Dienst musste dafür neu gestartet werden. Zwischen Aktivierung der Regel und INTERNET_ERROR lagen im Mittel 3,14 s. Bis der STM32 den Fehlerstatus erhielt, kamen durchschnittlich 4,22 s hinzu. Die End-to-End-Fehlerweitergabe dauerte damit im Mittel 7,36 s. Nach Aufhebung der Sperre lieferte die Wetterabfrage nach durchschnittlich 5,82 s wieder gültige Daten. Weitere 4,28 s später war NET=OK an den STM32 übertragen. Für die vollständige funktionale Wiederherstellung ergibt sich ein Mittelwert von 10,10 s. Die Standardabweichung von lediglich 0,14 s weist darauf hin, dass dieser Ablauf vor allem durch die periodischen Dienstzyklen bestimmt wird. Aufgrund des Sammelstatus lässt sich aus diesen Messungen nicht ableiten, ob der Prototyp einen Netzwerkausfall von einem Fehler des Wetterdienstes unterscheiden könnte.

4.5.3 G1: Verlust und Wiederkehr des GPS-Empfängers

Nach dem Trennen des USB-GPS setzte der GPS-Dienst den booleschen Status have_fix auf false. Der UART-Dienst kennzeichnete die Daten daraufhin mit GPS=ERR und MODE=LAST. Vom erkannten Geräteverlust bis zu dieser Weitergabe vergingen durchschnittlich 3,12 s. Nach dem Wiederanschluss stand ein gültiger Fix im Mittel nach 4,22 s zur Verfügung; die Umschaltung auf MODE=LIVE folgte nach weiteren 2,20 s. Insgesamt dauerte die funktionale Wiederherstellung im Mittel 6,42 s. Die Einzelwerte streuen deutlich. Ursachen sind die manuelle Fehlerinjektion, die jeweilige Phase der periodischen Dienstzyklen und der Hot-Start des GPS-Empfängers, bei dem noch gespeicherte Zeit- und Bahndaten die erneute Positionsbestimmung beschleunigen können.

4.5.4 C1: Unterbrechung der Verbindung zum STM32

Für C1 wurde die Sendeleitung des Raspberry Pi zum STM32 manuell getrennt. Alle fünf Unterbrechungen führten zu einer Verbindungsfehlermeldung. Ab der Bedienerbestätigung wurden dafür durchschnittlich 3,80 s benötigt. Nach dem Wiederverbinden traf das erste erfolgreiche PONG im Mittel nach 4,68 s ein. Der anschließend gesendete Wetterrahmen wurde bereits nach weiteren 0,05 s bestätigt. Die Gesamtzeiten lagen zwischen 0,13 s und 13,30 s. Fast die gesamte Streuung entstand vor dem ersten PONG, also durch die zeitliche Lage der Wiederverbindung innerhalb des Retry-Zyklus. Sobald der Heartbeat wieder funktionierte, verlief die Datenübertragung dagegen nahezu konstant.

4.5.5 H1: Firmware-Hänger und STM32-IWDG

Mit HANG wurde die Firmware gezielt angehalten; zuvor bestätigte der STM32 den Befehl durch HANGING. Das Boot-Banner erschien im Mittel 1,23 s später, ein PONG während der Bootdiagnose nach 3,35 s. Vom Absenden des HANG-Befehls bis

zum erneuten Applikationsbetrieb vergingen jeweils 5,70 s. Die Diagnose meldete in sämtlichen Durchläufen den IWDG als Reset-Ursache. Nach dem Wechsel in den Applikationsmodus funktionierte auch die PING/PONG-Kommunikation wieder. Dass die gerundeten Werte in allen fünf Messungen übereinstimmen, belegt die Reproduzierbarkeit innerhalb der Auflösung von 0,01 s.

4.5.6 S1 und S2: Hängender gegenüber abgestürztem GPS-Dienst

Der direkte Vergleich von S1 und S2 macht den Einfluss der Fehlerart sichtbar. Im Hängerfall existierte der Prozess weiter, lieferte jedoch keine Keepalive-Nachrichten. Systemd konnte erst nach Ablauf des Service-Watchdog-Timeouts reagieren; bis zur wiederhergestellten GPS-Funktion vergingen im Mittel 28,46 s. Nach SIGKILL war das Prozessende dagegen sofort erkennbar. Der Service-Manager musste nur die konfigurierte Neustartverzögerung abwarten und den neuen Prozess starten. Die funktionale Wiederherstellung dauerte deshalb im Mittel lediglich 5,76 s. Gegenüber dem Hänger verkürzte sich die Ausfallzeit um 22,70 s, also ungefähr um den Faktor 4,9.

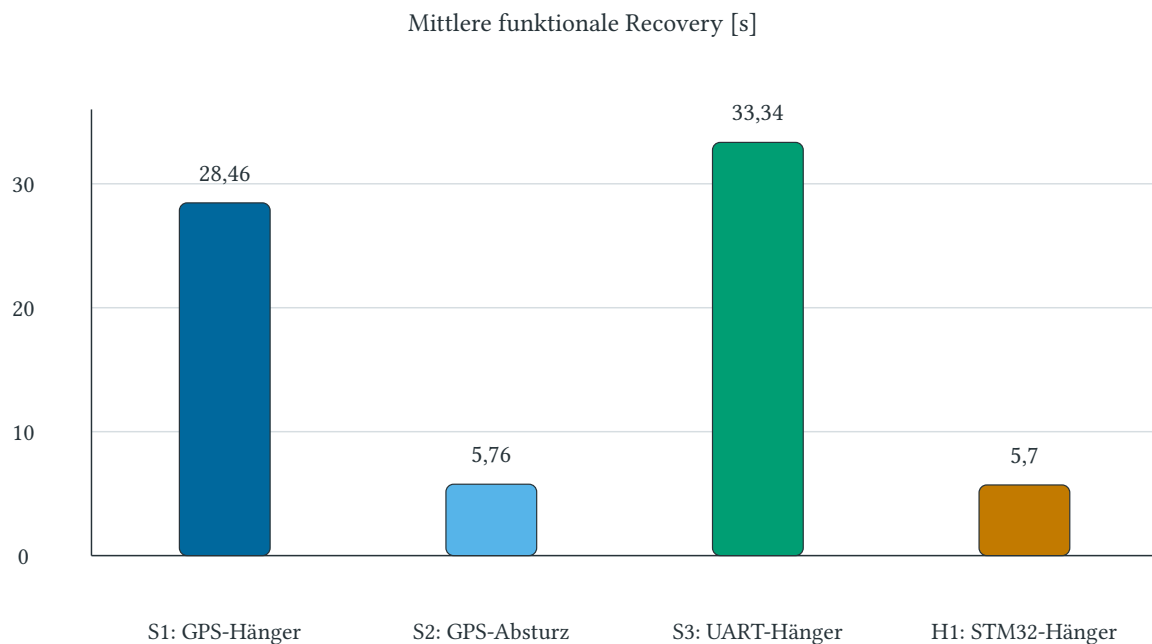


Abbildung 4.1: Vergleich der mittleren funktionalen Wiederherstellungszeiten

4.5.7 S3: Hänger des UART-Dienstes

Auch den angehaltenen UART-Dienst erkannte der systemd-Service-Watchdog in jedem Durchlauf. Bis zum Neustart vergingen durchschnittlich 33,22 s. Danach war die serielle Kommunikation nahezu unmittelbar verfügbar: Das erste PONG traf nach 0,04 s ein, die Bestätigung des Wetterrahmens nach weiteren 0,08 s. Daraus ergibt sich eine mittlere funktionale Wiederherstellungszeit von 33,34 s. Zusätzlich protokollierte der STM32 bei allen fünf Versuchen RPI_LINK_RECOVERED. Damit ist nicht nur der Neustart des Linux-Dienstes, sondern auch die erneute Funktionsfähigkeit des vollständigen Kommunikationspfads nachgewiesen.

4.5.8 U1 und U2: UART-Datenintegrität und Puffergrenzen

U1 bestätigte die Prüfsummenprüfung: Sämtliche fünf Rahmen mit bewusst falschem XOR-Wert wurden verworfen. Bis zur Antwort ERR:CHECKSUM vergingen durchschnittlich 0,010 s. Direkt danach akzeptierte der STM32 jeweils einen gültigen Rahmen und beantwortete auch den folgenden PING. Für U2 wurden Zeilen mit 140 Zeichen gesendet und damit die festgelegte Puffergrenze überschritten. Jeder Versuch erzeugte ERR:BUFFER; die Antwort erfolgte im Mittel nach 0,018 s. Anschließend blieben sowohl PING/PONG als auch die Verarbeitung eines gültigen Datenrahmens funktionsfähig. Die Fehlerbehandlung verwirft somit den betroffenen Rahmen, ohne den Empfänger dauerhaft zu desynchronisieren oder einen Reset auszulösen.

4.5.9 L1: Lokale Recovery des LCD-/I2C-Pfads

Das Trennen der SDA-Leitung führte in jedem Versuch zum Übergang von NORMAL nach DEGRADED; gleichzeitig änderte sich LCD_DRIVER von OK auf FAIL. Nach dem Wiederanschluss kehrte der Supervisor ohne Mikrocontroller-Reset in den Normalzustand zurück. Ab der Bedienerbestätigung lagen zwischen Eingriff und bestätigtem Supervisor-Zustand im Mittel 0,046 s für den Fehler sowie 0,044 s für die Recovery.

4.5.10 P1: Vollständiger Spannungsverlust

Der vollständige Spannungsverlust wurde in drei Durchläufen untersucht. Die externe Netzwerküberwachung erfasste Ausfallzeiten von 33 s, 32 s und 32 s. Der Mittelwert beträgt 32,33 s.

Tabelle 4.4: Externe Netzwerknichtverfügbarkeit bei vollständigem Spannungsverlust

Durchlauf	DOWN	UP	Dauer [s]
P1-01	13.07.2026 03:49:13	13.07.2026 03:49:46	33
P1-02	14.07.2026 00:53:00	14.07.2026 00:53:32	32
P1-03	14.07.2026 00:57:08	14.07.2026 00:57:40	32
		Mittelwert	32,33

Bei jedem Durchlauf änderte sich die Boot-ID, die eine einzelne Linux-Boot-Sitzung eindeutig kennzeichnet; anschließend starteten alle drei Dienste automatisch. GPS-Fix, Wetterabfrage und STM32-Kommunikation wurden ohne manuellen Eingriff wiederhergestellt. Für P1-02 und P1-03 konnten zusätzlich konsistente interne Boot-Messungen ausgewertet werden. Die vollständige Anwendungsbereitschaft lag bei 21,401 s und 21,102 s nach Beginn des Linux-Bootvorgangs. Der Mittelwert der beiden gültigen internen Messungen beträgt 21,252 s. Beim ersten Lauf war die interne Zeitmessung aufgrund einer nachträglichen Korrektur der Systemzeit nicht auswertbar. Die externe Messung blieb davon unbeeinflusst. Nach den abrupten Abschaltungen führte ext4 eine automatische Bereinigung verwaister Dateisystemeinträge durch; schwerwiegende I/O- oder Dateisystemfehler wurden nicht beobachtet.

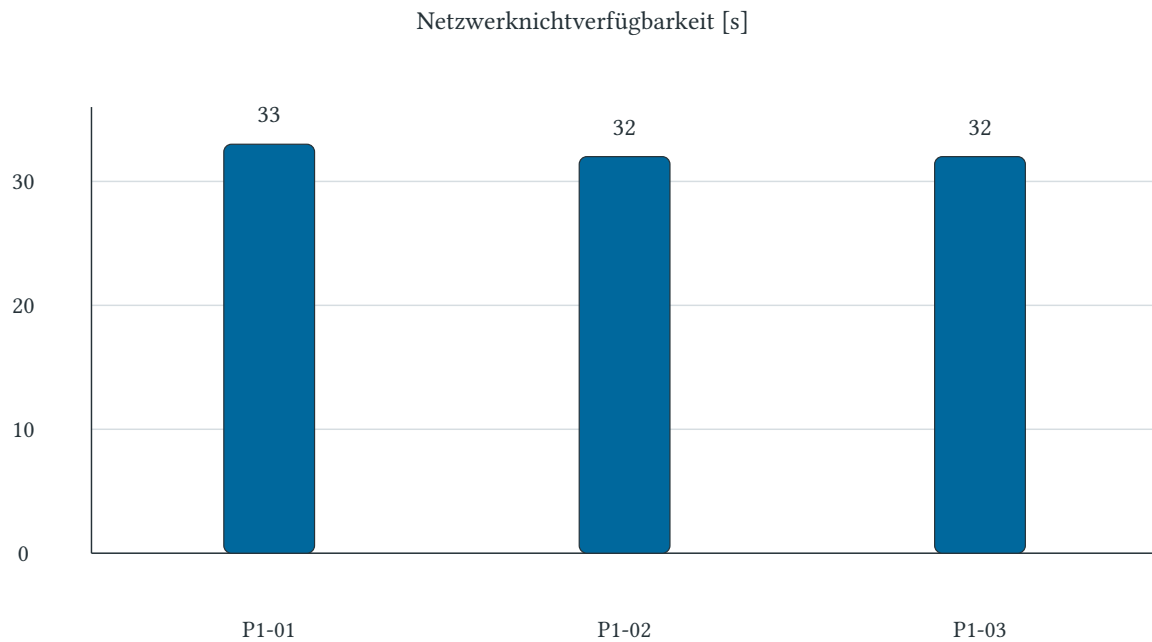


Abbildung 4.2: Extern gemessene Ausfallzeiten der drei P1-Durchläufe

4.5.11 P2: Kernel-Hänger und Hardware-Watchdog

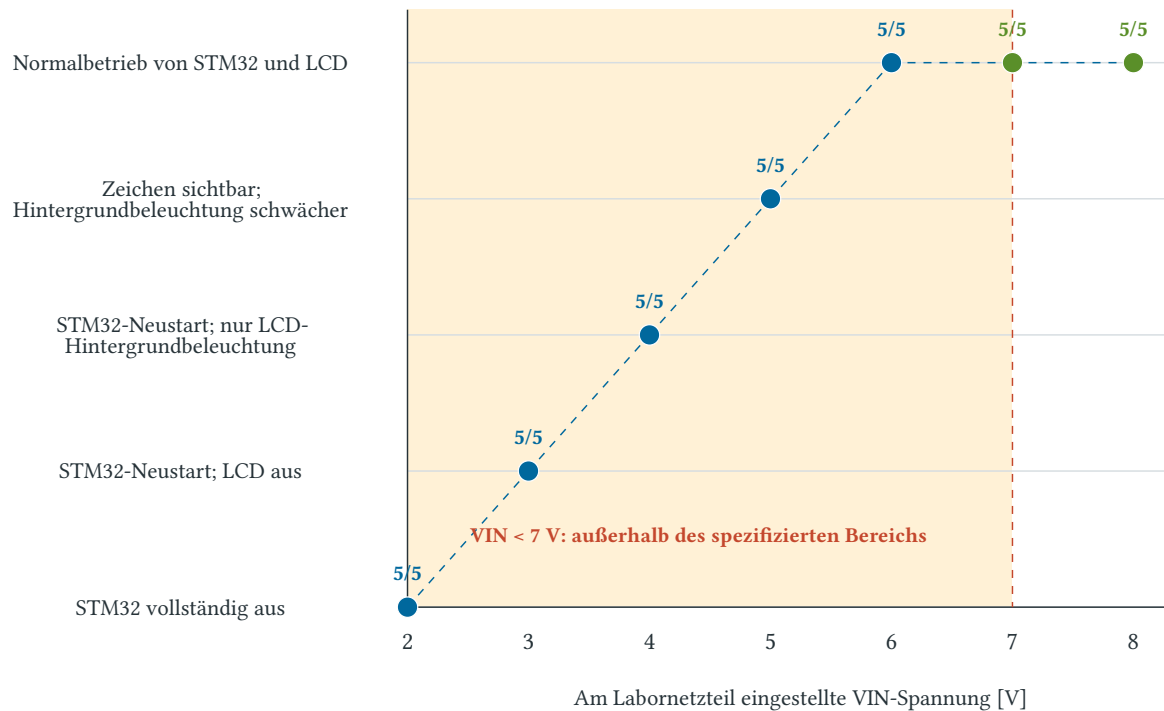
Der Raspberry Pi betrieb den BCM2835-Hardware-Watchdog mit einem von `wdctl` gemeldeten Timeout von 60 s. Der Fehler wurde als Kernel-Panic über Magic SysRq ausgelöst. Da `kernel.panic=0` einen automatischen Neustart des Kernels ausschloss, blieb als vorgesehener Recovery-Pfad nur der unabhängige Hardware-Watchdog.

Beide Durchläufe waren funktional erfolgreich: Die Boot-ID änderte sich, alle drei Dienste starteten erneut, und GPS, Wetterabfrage sowie STM32-Kommunikation standen wieder zur Verfügung. Für die zeitliche Auswertung liegt jedoch nur ein verwertbarer Durchlauf vor: Einzelmessung: 35 s; Der von `wdctl` gemeldete Watchdog-Timeout von 60 s beginnt nicht mit dem ersten fehlgeschlagenen externen Ping. Der Hardware-Watchdog wurde bereits vor der Fehlerinjektion periodisch aktualisiert; beim Auslösen der Kernel-Panic konnte deshalb ein Teil des aktuellen Watchdog-Intervalls bereits verstrichen sein. Die externe Messung setzte zudem erst beim Übergang von Netzwerkzustand UP zu DOWN ein. Der Einzelwert umfasst somit die verbleibende Watchdog-Zeit, den Hardware-Reset, den Linux-Boot und die Wiederherstellung der Netzwerkverbindung.

4.5.12 P3: Kontrollierter Unterspannungstest NUCLEO-L432KC mit LCD

Die NUCLEO-L432KC-Platine wurde über VIN und GND mit dem Labornetzteil versorgt. Nach Herstellerangabe ist für VIN ein Eingangsbereich von 7 V bis 12 V vorgesehen [24, Abschn. 7.4 und Tab. 5]. Die Werte unterhalb von 7 V lagen damit gezielt außerhalb des spezifizierten Betriebsbereichs. Die Spannung wurde in 1-V-Schritten

von 8 V auf 2 V abgesenkt. Der vollständige Ablauf wurde fünfmal wiederholt; in allen fünf Durchläufen wurde dasselbe qualitative Verhalten beobachtet.



Qualitative Zustandsklassen (ordinal). Die gestrichelte Verbindung zeigt nur die Reihenfolge der Messstufen und bestimmt keine Schwellenspannung.

Abbildung 4.3: Qualitative Degradationsstufen in Abhängigkeit von der am Labornetzteil eingestellten VIN-Spannung

Abbildung 4.3 stellt die beobachtete Degradationsfolge auf zwei Achsen dar. Auf der x-Achse steht die am Labornetzteil eingestellte VIN-Spannung. Für die y-Achse wurden die qualitativen Beobachtungen in fünf ordinal geordnete Zustandsklassen überführt. Die Abstände zwischen diesen Klassen besitzen keine metrische Bedeutung. Die Beschriftung „5/5“ kennzeichnet, dass der jeweilige Zustand in allen fünf Durchläufen beobachtet wurde.

Tabelle 4.5: Reproduzierbares Unterspannungsverhalten der NUCLEO-L432KC-Platine

VIN	Beobachtung	Durchläufe
8–6 V	STM32 und LCD arbeiteten ohne sichtbare Einschränkungen; Zeichen und Hintergrundbeleuchtung waren normal.	5/5
5 V	Die Zeichen blieben sichtbar; die Hintergrundbeleuchtung wurde erkennbar schwächer.	5/5
4 V	Die Hintergrundbeleuchtung blieb aktiv, Zeichen wurden nicht mehr dargestellt; gleichzeitig wurde ein STM32-Neustart beobachtet.	5/5
3 V	Ein weiterer STM32-Neustart wurde beobachtet; das LCD einschließlich Hintergrundbeleuchtung war vollständig aus.	5/5
2 V	Der STM32 stellte den Betrieb vollständig ein.	5/5

Die Beobachtungen zeigen keinen abrupten Übergang von vollständiger Funktion zu vollständigem Ausfall, sondern eine reproduzierbare Degradationsfolge: zunächst verringerte sich die Helligkeit, anschließend fiel die Zeichendarstellung aus, danach traten Neustarts und schließlich der vollständige Stillstand ein. Ein solcher Bereich zwischen normaler Funktion und Abschaltung entspricht konzeptionell der in der Literatur beschriebenen Unterspannungszone, in der Komponenten bereits unzuverlässig arbeiten können [26, S. 15–16].

Raspberry Pi bei 4,50 V

Beim Raspberry Pi wurde bei einer am Labornetzteil eingestellten Spannung von 4,50 V in Abständen von 0,5 s `vcgencmd get_throttled` aufgerufen. Die erfasste Ausgabe lautete `throttled=0x50005`.

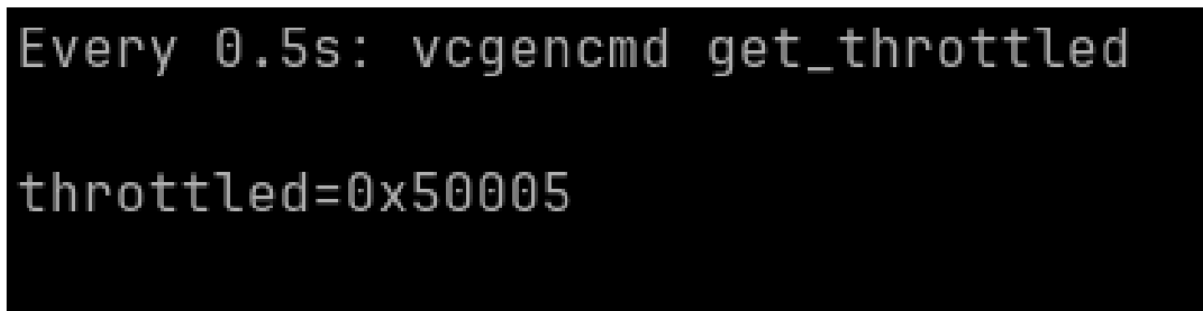


Abbildung 4.4: Raspberry-Pi-Status bei 4,50 V: `throttled=0x50005`

Nach der offiziellen Raspberry-Pi-Dokumentation stehen die gesetzten Bits 0 und 2 für aktuell erkannte Unterspannung und aktuell aktive Drosselung; die Bits 16 und 18 speichern, dass beide Zustände seit dem letzten Start bereits aufgetreten sind [25, Abschn. “`get_throttled`”]. Die Messung belegt damit eine zum Aufnahmezeitpunkt aktive Unterspannung und Drosselung.

4.6 Zusammenfassung der Messergebnisse

Innerhalb der beschriebenen Versuchsbedingungen wurden alle fünfmal wiederholten funktionalen Recovery-Testfälle erfolgreich abgeschlossen. Die zentralen Einzel- und Aggregatwerte sind in Tabelle 4.3 zusammengefasst. Für P1 lag die mittlere externe Netzwerknichtverfügbarkeit bei 32,33 s; für P2 liegt ausschließlich die oben ausgewiesene Einzelmessung von 35 s vor. P3 zeigte in fünf von fünf STM32-Durchläufen dieselbe stufenweise Degradation; die Raspberry-Pi-Messung belegte bei 4,50 V eine aktive Unterspannung und Drosselung. Die Interpretation der Unterschiede, die Beantwortung der Forschungsfragen und die Grenzen der Übertragbarkeit folgen in Kapitel 5.

5 Diskussion

Die folgenden Abschnitte interpretieren die in Kapitel 4 berichteten Messwerte. Aussagen zur Wirksamkeit beziehen sich auf die vorliegende Arbeit, die beschriebenen Fehlerklassen und die durchgeführten Versuchsdurchläufe.

Die folgende Tabelle fasst zunächst zusammen, welche Recovery-Ebene den einzelnen untersuchten Fehlerklassen zugeordnet wurde und welche Aussage daraus im Rahmen dieser Arbeit abgeleitet wird.

Tabelle 5.1: Zuordnung der untersuchten Fehlerklassen zu Recovery-Ebenen

Fehlerklasse	Strategie	Interpretation in der Arbeit
Ungültige Prüfsumme / Pufferüberlauf	Rahmen verwerfen bzw. Empfänger synchronisieren	Fehler bleibt auf Protokollebene isoliert.
LCD-/I2C-Ausfall	Lokale Komponenten-Recovery	Anzeigeausfall stoppt den übrigen Datenpfad nicht.
GPS-Verlust	Gekennzeichneter Fallback und Retry	Letzte zulässige Position wird als MODE=LAST gekennzeichnet.
Blockierter HTTPS-Wetterdatenpfad	Degradierter Zustand und periodischer Retry	Kein gültiger Wetterdatensatz; Ursache innerhalb des Datenpfads bleibt unbestimmt.
Prozessabsturz	systemd-Restart	Prozessende ist unmittelbar sichtbar.
Prozess-Hänger	systemd-Service-Watchdog	Fehlender Funktionsfortschritt muss über ausbleibende Keepalives erkannt werden.
STM32-Firmware-Hänger	IWDG-Reset	Firmware kann keine lokale Recovery mehr ausführen.
Linux-Kernel-Hänger	Hardware-Watchdog	Linux-Prozesse im Benutzerraum und systemd sind nicht mehr handlungsfähig.
Vollständiger Spannungsverlust	Automatischer Boot und Dienststart	Recovery beginnt erst nach Rückkehr der Versorgung.

5.1 Beantwortung der Forschungsfragen

5.1.1 F1: Strukturierung und geeignete Fehlerdomäne

Die Versuche sprechen dafür, Fehler auf der niedrigsten Ebene zu behandeln, auf der noch eine funktionsfähige Ausführungsbasis vorhanden ist. Datenfehler werden im Parser verworfen, ein LCD-Fehler im Mikrocontroller lokal behandelt, Prozessfehler durch systemd und ein vollständiger Linux-Hänger durch den Hardware-Watchdog. Die Architektur folgt damit dem Recovery-orientierten Prinzip der Partitionierung und einer mehrstufigen Verteidigung [2, S. 5 und 8]. Die Trennung der Raspberry-Pi-Anwendung in drei Dienste war dafür zentral. Während des blockierten HTTPS-Wetterdatenpfads blieben GPS-Erfassung und UART aktiv. Beim Neustart des GPS-

Dienstes musste weder die Wetterlogik noch der serielle Link zurückgesetzt werden. Die Komponentengrenzen wurden somit zugleich zu Recovery-Grenzen.

5.1.2 F2: Eingriffstiefe und Wiederherstellungszeit

Lokale Datenfehler wurden im Millisekundenbereich behandelt. Ein Dienstabsturz benötigte im Mittel 5,76 s, ein Dienst-Hänger 28,46 s. Der STM32-Firmware-Hänger führte nach 5,70 s zurück in den Anwendungsmodus. Vollständige Neustarts verursachten mit etwa 32 s bis 35 s die größte beobachtete Netzwerkunterbrechung. Diese Abstufung zeigt im untersuchten Aufbau, dass die Eingriffstiefe einen praktischen Einfluss auf die Ausfallzeit hat. Feingranulare Recovery erhält funktionsfähige Zustände und entspricht dem Grundgedanken eines Microreboots [6, S. 31–32]. Ein vollständiger Reset bleibt dennoch unverzichtbar, wenn die betroffene Fehlerdomäne selbst keine Recovery mehr ausführen kann.

5.1.3 F3: Absturz, Hänger, Firmware-Hänger und System-Hänger

Ein Prozessabsturz ist für systemd sofort sichtbar. Die gemessene Wiederherstellung wurde deshalb im Wesentlichen von RestartSec=5s bestimmt. Ein mit SIGSTOP erzeugter Hänger ließ den Prozess dagegen formal bestehen. Erst das Ausbleiben von WATCHDOG=1 führte zum Timeout. Diese Differenz zeigt, weshalb eine Restart-Policy allein nicht gegen Hänger schützt. Beim STM32-Hänger blockierte die vollständige Firmwareausführung; der unabhängig getaktete IWDG blieb als Recovery-Basis erhalten. Beim Kernel-Panic konnte auch systemd nicht mehr reagieren. Erst der BCM2835-Hardware-Watchdog stellte das System wieder her. Die Versuche bilden damit eine Hierarchie zunehmend schwerer Fehlerdomänen ab.

5.1.4 F4: Automatische Rückkehr zum definierten Zustand

Innerhalb des untersuchten Umfangs erreichten alle wiederholten funktionalen Recovery-Tests den vordefinierten Endzustand. Beim GPS-Verlust und beim blockierten HTTPS-Wetterdatenpfad bedeutete dies nicht immer sofort den vollständigen Sollzustand: Das System wechselte zunächst in einen transparenten degradierten Zustand und kehrte nach Wegfall der Störung zu LIVE beziehungsweise NET=OK zurück. Auch Firmware- und Systemneustarts benötigten nach dem Reset keinen manuellen Start der Anwendung. Die Aussage ist auf die getesteten Fehlerklassen und die kleine Stichprobe beschränkt. Fünf erfolgreiche Durchläufe pro risikoarmem Testfall zeigen eine reproduzierbare Funktion innerhalb des untersuchten Aufbaus.

5.2 Dienst-Hänger gegenüber Dienst-Absturz

Der Vergleich von S1 (durch SIGSTOP angehaltener GPS-Dienst) und S2 (durch SIGKILL beendeter GPS-Dienst) ist der deutlichste quantitative Unterschied. Der hängende GPS-Dienst benötigte im Mittel 28,46 s, der abgestürzte Dienst 5,76 s. Der Hänger war damit ungefähr 4,9-mal langsamer wiederhergestellt. Der Unterschied ist kein Mangel der Restart-Policy, sondern eine Konsequenz der fehlenden unmittelbaren Beobachtbarkeit eines Hängers. Die erste S1-Wiederholung dauerte länger als die folgenden

Läufe, weil die Fehlerinjektion zu einem anderen Zeitpunkt relativ zum letzten Keepalive erfolgte. Ein Watchdog-Timeout wird ab dem letzten akzeptierten Lebenszeichen berechnet und nicht ab dem extern beobachteten SIGSTOP.

5.3 Nutzen des degradierten Betriebs

Der GPS-Fallback und die LCD-Recovery zeigen, dass ein System trotz eines Teilausfalls noch einen sinnvollen Dienst bereitstellen kann. Bei fehlendem GPS-Empfänger wurden letzte bekannte Daten nur mit ausdrücklicher Kennzeichnung GPS=ERR;MODE=LAST weitergegeben. Der LCD-Ausfall beeinträchtigte nicht die UART- und Supervisorfunktion. Dadurch blieb das System diagnostizierbar und konnte die Wiederkehr der Komponente erkennen. Ein degradiertes Modell verlangt jedoch klare Semantik. Veraltete Daten dürfen nicht als aktuell erscheinen, und kritische Fehler dürfen nicht als nicht-kritisch eingestuft werden. In der Arbeit begrenzt MAX_POSITION_AGE_SECONDS die Wiederverwendung einer letzten Position.

5.4 Abgrenzung von Netzwerkpfad und Wetterdienst

N2 belegt, dass der Prototyp einen blockierten HTTPS-Wetterdatenpfad als degradierten Zustand weitergibt und nach Freigabe automatisch zu gültigen Wetterdaten zurückkehrt. Der Versuch belegt dagegen weder eine allgemeine Netzwerkd Diagnose noch die Erkennung eines isolierten Ausfalls des Wetterdienstes. Das Blockieren von Port 443 und der gemeinsame Status NET=ERR lassen die Ursache innerhalb des externen Datenpfads offen.

Für eine ursachenspezifische Recovery wären zwei orthogonale Gesundheitszustände erforderlich. Der Zustand des Netzwerkpfsds müsste aus lokalen Schnittstellen-, Adress- und Routinginformationen sowie geeigneten Verbindungsdiagnosen abgeleitet werden. Ist dieser Pfad nachweislich gestört, kommen Wiederverbindungsversuche und bei bestätigtem lokalem Fehler eine Reinitialisierung der Netzwerkkonfiguration in Betracht. Der Wetterdienst kann dagegen nur bewertet werden, wenn der Netzwerkpfad grundsätzlich verfügbar ist. Dienstfehler, Ratenbegrenzungen oder ungültige Antworten erfordern keinen Netzwerkreset, sondern einen dienstspezifischen Retry mit Backoff und das Verwerfen beziehungsweise Kennzeichnen der betroffenen Daten. Ist der Netzwerkpfad ausgefallen, bleibt der Zustand des Wetterdienstes unbekannt. Eine dauerhafte externe Störung verbleibt im Zustand DEGRADED und löst für sich allein keinen Plattform-Reset aus. Der Prototyp setzt diese Differenzierung noch nicht um; sie bildet daher eine Implementierungs- und Evaluationsgrenze.

5.5 Lokale Recovery und Zustandsverlust

Eine Recovery setzt Zustände zurück. Je größer die Recovery-Domäne, desto mehr korrekter Zustand geht verloren. Ein vollständiger STM32-Reset hätte das LCD ebenfalls reinitialisiert, gleichzeitig aber UART-Puffer, Diagnosezustände und laufende Kommunikation unterbrochen. Die lokale I2C-/LCD-Reinitialisierung vermied diesen

Kollateraleffekt. Dasselbe Prinzip gilt für Linux-Dienste. Die Zustandsdateien in tmpfs entkoppeln Prozesse, sind aber bewusst flüchtig. Nach einem vollständigen Reboot wird kein alter Zustand übernommen; dafür muss der Datenpfad neu aufgebaut werden. Diese Entscheidung bevorzugt Datenfrische gegenüber Persistenz.

5.6 Datenintegrität und Grenzen der XOR-Prüfsumme

U1 (absichtlich falsche Prüfsumme) und U2 (Überschreitung der Rahmenlänge) zeigen innerhalb der durchgeführten Versuche, dass die implementierten Grenzen greifen: Fehlerhafte Prüfsummen und übergroße Rahmen wurden in allen Wiederholungen abgelehnt, und die Kommunikation blieb anschließend synchron. Damit wurde der Schutz gegen die gezielt injizierten einfachen Fehler im Versuchssystem demonstriert. Die XOR-Prüfsumme darf jedoch nicht überinterpretiert werden. Sie erkennt beispielsweise bestimmte Mehrbitfehler nicht und bietet keinen Schutz gegen absichtliche Manipulation. Untersuchungen zu eingebetteten Prüfsummen zeigen die deutlich bessere Fehlererkennungsleistung geeigneter CRCs [12, S. 59–60 und 67–69, 13, S. 145–146].

5.7 Trade-off der Watchdog-Zeiten

Die Timeout-Werte beeinflussen Verfügbarkeit und Fehlalarmrisiko. Ein kürzerer Service-Timeout hätte S1 und S3 beschleunigt, könnte aber bei langsamen Gerätezugriffen oder hoher Systemlast unnötige Neustarts erzeugen. Die Arbeit verwendet 30 s, während die normalen Dienstperioden bei 5 s beziehungsweise 10 s liegen. Für ein Produkt müssten Worst-Case-I/O-Zeiten, CPU-Last, Netzwerk-Timeouts und zulässige maximale Ausfallzeit gemeinsam betrachtet werden. Die Notwendigkeit, den Timeout aus dem zeitlichen Verhalten der gesamten überwachten Software abzuleiten, wird auch in der Watchdog-Literatur hervorgehoben [10, S. 12] [11, S. 16].

Auch beim Hardware-Watchdog dürfen Konfigurationswert und extern beobachtete Ausfallzeit nicht gleichgesetzt werden. wdctl meldete einen Timeout von 60 s; für P2 liegt jedoch nur eine zeitlich auswertbare externe Einzelmessung von 35 s vor. Der Watchdog lief bereits vor der Kernel-Panic und wurde periodisch aktualisiert. Zum Zeitpunkt der Fehlerinjektion konnte daher ein Teil des aktuellen 60 s-Intervalls bereits verstrichen sein. Die externe Messung begann außerdem erst mit dem ersten fehlgeschlagenen Ping und nicht exakt mit der Kernel-Panic. Der Wert von 35 s enthält die verbleibende Watchdog-Zeit, Reset, Bootvorgang und Netzwerkinitialisierung. Er beschreibt ausschließlich die in diesem einen Durchlauf beobachtete End-to-End-Netzwerknichtverfügbarkeit und nicht die reine Watchdog-Auslösezeit.

5.8 Unterspannungsverhalten

Der ergänzende P3-Test zeigt, dass die Energieversorgung eine eigene Fehlerdomäne bildet und dass verschiedene Komponenten nicht gleichzeitig ausfallen müssen. Beim Nucleo-Aufbau war die verringerte LCD-Helligkeit das erste sichtbare Symptom.

Danach fiel die Zeichendarstellung aus, während die Hintergrundbeleuchtung noch aktiv blieb; erst bei weiter abgesenkter Eingangsspannung erlosch das LCD vollständig und der STM32 stellte schließlich den Betrieb ein. Dieses Verhalten entspricht einem degradierten Systemzustand. Der Raspberry-Pi-Wert 0x50005 ergänzt die rein visuelle Beobachtung um einen softwareseitigen Statusnachweis. Die gesetzten aktuellen und gespeicherten Bits zeigen, dass Unterspannung und Drosselung nicht nur früher aufgetreten waren, sondern zum Abfragezeitpunkt aktiv waren [25, Abschn. “get_throttled”].

Die Arbeiten Mementos und DINO verdeutlichen darüber hinaus, dass eine robuste Behandlung wiederkehrender Versorgungsausfälle nicht beim Reset endet: Je nach Persistenzmodell müssen Fortschritt und konsistente Zustandsgrenzen ausdrücklich abgesichert werden [27, S. 159–163; 28, S. 575–579]. Für die vorliegende Arbeit ist daraus vor allem abzuleiten, dass ein zukünftiger Power-Monitor vor Eintritt des unsicheren Bereichs einen definierten Zustand herstellen und dessen Erfolg nach dem Neustart verifizieren sollte.

5.9 Validität und Messunsicherheit

Interne Validität betrifft die Frage, ob die beobachteten Effekte tatsächlich auf die jeweilige Fehlerinjektion zurückgeführt werden können. Konstruktvalidität bewertet, ob die gewählten Messendpunkte die behauptete funktionale Wiederherstellung angemessen abbilden. Externe Validität beschreibt die Übertragbarkeit auf andere Hardware-, Software- und Umgebungsbedingungen. Als Messunsicherheiten werden hier insbesondere die Abtastintervalle, manuelle Bedienerbestätigungen, unterschiedliche Phasen der Dienstzyklen, kleine Stichproben und die einzelne auswertbare P2-Zeitmessung berücksichtigt.

5.9.1 Interne Validität

Automatisierte Tests verwendeten monotone Zeitmessung und Journal-Cursor, um alte Ereignisse auszuschließen. Bei manuellen Leitungsfehlern beginnt die Messung nach Bedienerbestätigung.

5.9.2 Konstruktvalidität

Die funktionale Recovery wurde nicht nur über den systemd-Zustand active (running) festgestellt, der lediglich die Existenz eines laufenden Prozesses belegt. Ein GPS-Dienst musste einen neuen Fix liefern, der UART-Dienst musste PONG und OK erreichen, und der STM32 musste in den als APP_RUNTIME protokollierten Applikationsmodus zurückkehren. Diese Endpunkte bilden die Nutzfunktion besser ab als reine Prozesszustände.

5.9.3 Externe Validität

Die Ergebnisse gelten zunächst für den in dieser Arbeit beschriebenen Versuchsaufbau aus der verwendeten Hardware, dem installierten Betriebssystem und der konkreten

Netzwerktopologie. Andere SD-Karten, Kernelversionen, GPS-Empfangsbedingungen oder Wetterdienste können andere Zeiten und Fehlerverläufe erzeugen.

5.10 Gesamtbewertung

Das Gesamtkonzept erfüllt die Zielsetzung für den in dieser Arbeit untersuchten Aufbau mit der genannten Einschränkung des aggregierten Wetterdatenstatus. Es trennt die übrigen Fehlerdomänen, macht Zustände beobachtbar und eskaliert die Recovery nach Kritikalität. Keine einzelne Ebene könnte alle Fehler abdecken: systemd kann einen Kernel-Hänger nicht beheben, der IWDG keinen Ausfall eines externen Datenpfads und ein Hardware-Reboot wäre für einen Prüfsummenfehler unverhältnismäßig. Die wichtigste Entwurfserkenntnis lautet daher: Ein robustes heterogenes System benötigt nicht den größtmöglichen Reset, sondern die kleinste hinreichende Recovery-Maßnahme plus eine unabhängige Eskalationsstufe für den Fall, dass diese Maßnahme selbst nicht mehr ausgeführt werden kann.

6 Fazit und Ausblick

6.1 Fazit

Diese Arbeit hat Watchdog- und Recovery-Strategien für ein heterogenes eingebettetes System aus Raspberry Pi und STM32 untersucht. Als Testanwendung wurde ein vollständiger Datenpfad aus GPS-Empfang, Wetterabfrage, UART-Kommunikation und LCD-Anzeige aufgebaut. Der Beitrag liegt nicht in der Wetteranwendung selbst, sondern in der Zuordnung von Fehlerklassen zu abgestuften Recovery-Mechanismen und deren experimenteller Bewertung. Die implementierte Architektur behandelt Datenfehler lokal, reinitialisiert nicht kritische Peripherie, verwendet bei externen Ausfällen einen degradierten Zustand, startet einzelne Linux-Dienste neu und eskaliert erst bei Firmware- oder System-Hängern zu IWDG-Reset beziehungsweise Hardware-Reboot. Die Aufteilung in getrennte systemd-Dienste begrenzt Fehlerdomänen; der STM32-Supervisor verknüpft den IWDG-Refresh mit dem Fortschritt kritischer Subsysteme. Innerhalb der untersuchten Stichprobe waren alle wiederholten funktionalen Recovery-Testfälle in fünf von fünf Durchläufen erfolgreich. Besonders deutlich war der Unterschied zwischen Prozess-Hänger und Prozessabsturz: Der hängende GPS-Dienst benötigte im Mittel 28,46 s, der abgestürzte Dienst 5,76 s bis zur funktionalen Wiederherstellung. Der STM32 kehrte nach einem absichtlichen Firmware-Hänger in 5,70 s zum Anwendungsmodus zurück. Der vollständige Spannungsverlust führte zu einer mittleren externen Netzwerk-Nichtverfügbarkeit von 32,33 s. Der ergänzende Unterspannungstest zeigte bei der Nucleo-Platine in fünf von fünf Durchläufen dieselbe stufenweise Degradation; auf dem Raspberry Pi belegte der Statuswert 0x50005 eine bei der untersuchten Versorgungssituation aktive Unterspannung und Drosselung. Für den in dieser Arbeit untersuchten Aufbau und die durchgeführten Versuche lassen sich die Forschungsfragen wie folgt beantworten: Fehler müssen auf der kleinsten noch funktionsfähigen Ebene erkannt und behandelt werden; feingranulare Recovery reduziert Zustandsverlust und Ausfallzeit; Hänger benötigen explizite Fortschrittsüberwachung; und die untersuchte unabhängige Watchdog-Kette führte bei den injizierten schweren Fehlern zur automatischen Rückkehr in einen definierten Zustand. Eine einzige Watchdog-Strategie ist für ein solches System nicht ausreichend.

6.2 Ausblick

Für eine Weiterentwicklung bieten sich an:

- quantitative Brown-out- und PVD-Versuche mit definierter Spannungsrampe, Oszilloskopmessung direkt an den Versorgungsschienen, Laststromaufzeichnung und protokollierten RCC-Resetflags,
- Langzeit- und Stresstests mit deutlich größerer Stichprobe,
- Einsatz eines Window Watchdogs zur Erkennung zu früher Lebenszeichen oder eines separaten externen Watchdog-Bausteins als zusätzliche unabhängige Überwachung,
- CRC für eine stärkere Fehlererkennung, Sequenznummern zur Erkennung verlorener oder wiederholter Rahmen und explizite Nachrichtenalterung gegen die Verwendung veralteter Daten im UART-Protokoll,

- getrennte Gesundheitszustände für Netzwerkpfad und Wetterdienst mit ursachenspezifischer Diagnose und Recovery sowie separaten Fehlerinjektionstests für Netzverlust, HTTP-Dienstfehler und ungültige API-Antworten,
- persistente Reset- und Fehlerzähler in einem verschleißbewusst beschriebenen Speicher,
- Read-only-Root-Dateisystem, Overlay-Dateisystem oder Energiepuffer, um unvollständige Schreibvorgänge und Dateisystemschäden auf der SD-Karte zu begrenzen,
- abgesicherter Bootloader mit signierten Firmware-Updates zur Authentizitätsprüfung und Rollback zur Rückkehr auf die letzte funktionsfähige Firmware,
- zentrale Telemetrie zur Sammlung von Zustandsdaten, automatisierte Alarmierung und Trendanalyse zur frühzeitigen Erkennung wiederkehrender Fehler,
- systematische Fault-Injection für Speicher-, Ressourcen- und Timingfehler.

Diese Erweiterungen würden die Übertragbarkeit auf produktive Systeme erhöhen, während das in der Arbeit untersuchte Grundprinzip erhalten bliebe: lokale Recovery zuerst, kontrollierte Eskalation nur bei Bedarf und unabhängige Überwachung für den Ausfall der jeweils darunterliegenden Ebene.

Literaturverzeichnis

- [1] Algirdas Avizienis u. a. „Basic Concepts and Taxonomy of Dependable and Secure Computing“. In: IEEE Transactions on Dependable and Secure Computing 1.1 (2004), S. 11–33. doi: 10.1109/TDSC.2004.2.
- [2] David A. Patterson u. a. Recovery-Oriented Computing (ROC): Motivation, Definition, Techniques, and Case Studies. Techn. Ber. UCB/CSD-02-1175. University of California, Berkeley, 2002 <https://digicoll.lib.berkeley.edu/record/137942> (besucht am 17.07.2026).
- [3] Linux Kernel Documentation Project. The Linux Watchdog Driver API. <https://docs.kernel.org/watchdog/watchdog-api.html> (besucht am 17.07.2026).
- [4] RM0394: STM32L41xxx/42xxx/43xxx/44xxx/45xxx/46xxx Advanced Arm-based 32-bit MCUs. Reference Manual. STMicroelectronics, Rev. 5, Dezember 2024. https://www.st.com/resource/en/reference_manual/rm0394-stm32l41xxx42xxx43xxx44xxx45xxx46xxx-advanced-armbased-32bit-mcus-stmicroelectronics.pdf (besucht am 19.09.2026).
- [5] Brian Randell. „System Structure for Software Fault Tolerance“. In: IEEE Transactions on Software Engineering SE-1.2 (1975), S. 220–232. doi: 10.1109/TSE.1975.6312842.
- [6] George Candea u. a. „Microreboot—A Technique for Cheap Recovery“. In: Proceedings of the 6th Symposium on Operating Systems Design and Implementation (OSDI ’04). USENIX Association, 2004, S. 31–44. https://www.usenix.org/legacy/event/osdi04/tech/full_papers/candea/candea.pdf (besucht am 17.07.2026).
- [7] systemd Project. systemd.service—Service Unit Configuration. <https://www.freedesktop.org/software/systemd/man/latest/systemd.service.html> (besucht am 17.07.2026).
- [8] systemd Project. sd_notify—Notify Service Manager About Start-up Completion and Other Service Status Changes. https://www.freedesktop.org/software/systemd/man/latest/sd_notify.html (besucht am 17.07.2026).
- [9] DS11451: STM32L432KB/STM32L432KC Datasheet. STMicroelectronics, 2018. <https://www.st.com/resource/en/datasheet/stm32l432kc.pdf> (besucht am 17.07.2026).
- [10] Michael J. Pont und Royan H. L. Ong. „Using Watchdog Timers to Improve the Reliability of TTCS Embedded Systems: Seven New Patterns and a Case Study“. In: Proceedings of VikingPLoP 2002. Denmark, 2002. <https://www.cs.le.ac.uk/SoftSD/resources/viking02.pdf> (besucht am 20.07.2026).
- [11] Jacob Beningo. A Review of Watchdog Architectures and their Application to CubeSats. University of Michigan, 2010. <https://www.beningo.com/wp-content/uploads/images/Papers/WatchdogArchitectureReview.pdf> (besucht am 20.07.2026).
- [12] Theresa C. Maxino und Philip J. Koopman. „The Effectiveness of Checksums for Embedded Control Networks“. In: IEEE Transactions on Dependable and Secure Computing 6.1 (2009), S. 59–72. doi: 10.1109/TDSC.2007.70216.
- [13] Philip Koopman und Tridib Chakravarty. „Cyclic Redundancy Code (CRC) Polynomial Selection for Embedded Networks“. In: International Conference on Dependable Systems and Networks. 2004, S. 145–154. doi: 10.1109/DSN.2004.1311885.
- [14] UM10204: I2C-bus Specification and User Manual. NXP Semiconductors, 2021. <https://www.nxp.com/docs/en/user-guide/UM10204.pdf> (besucht am 17.07.2026).
- [15] PCF8574; PCF8574A: Remote 8-bit I/O Expander for I2C-bus with Interrupt. Rev. 5. NXP Semiconductors, 2013. https://www.nxp.com/docs/en/data-sheet/PCF8574_PCF8574A.pdf (besucht am 20.07.2026).

- [16] u-blox 7 Receiver Description Including Protocol Specification V14. GPS.G7-SW-12001-B1. u-blox AG, 2018. https://content.u-blox.com/sites/default/files/products/documents/u-blox7-V14_ReceiverDescriptionProtocolSpec_%28GPS.G7-SW-12001%29_Public.pdf (besucht am 17.07.2026).
- [17] Open-Meteo. Weather Forecast API Documentation. <https://open-meteo.com/en/docs> (besucht am 17.07.2026).
- [18] Raspberry Pi Ltd. Configure UARTs. <https://www.raspberrypi.com/documentation/computers/configuration.html#configuring-uarts> (besucht am 18.07.2026).
- [19] Serielle Schnittstelle I2C für LCD Displays—Datenblatt. AZ-Delivery Vertriebs GmbH. https://cdn.shopify.com/s/files/1/1509/1638/files/Serielle_Schnittstelle_I2C_fur_LCD_Displays_Datenblatt.pdf (besucht am 23.07.2026).
- [20] systemd Project. systemd-system.conf—System and Service Manager Configuration. <https://www.freedesktop.org/software/systemd/man/latest/systemd-system.conf.html> (besucht am 17.07.2026).
- [21] Arm Limited. CMSIS-Core Documentation. https://arm-software.github.io/CMSIS_6/latest/Core/index.html (besucht am 17.07.2026).
- [22] Mei-Chen Hsueh, Timothy K. Tsai und Ravishankar K. Iyer. „Fault Injection Techniques and Tools“. In: Computer 30.4 (1997), S. 75–82. doi: 10.1109/2.585157.
- [23] Python Software Foundation. time.monotonic—Monotonic Clock. <https://docs.python.org/3/library/time.html#time.monotonic> (besucht am 17.07.2026).
- [24] STMicroelectronics. UM1956: STM32 Nucleo-32 Boards (MB1180), User Manual. Rev. 6, 2025. https://www.st.com/resource/en/user_manual/um1956-stm32-nucleo32-boards-mb1180-stmicroelectronics.pdf (besucht am 02.08.2026).
- [25] Raspberry Pi Ltd. Raspberry Pi OS Documentation—vcgencmd, get_throttled. <https://www.raspberrypi.com/documentation/computers/os.html#vcgencmd> (besucht am 02.08.2026).
- [26] Domenico Balsamo, Alex S. Weddell, Geoff V. Merrett, Bashir M. Al-Hashimi, Davide Brunelli und Luca Benini. „Hibernus: Sustaining Computation During Intermittent Supply for Energy-Harvesting Systems“. In: IEEE Embedded Systems Letters 7.1 (2015), S. 15–18. doi: 10.1109/LES.2014.2371494.
- [27] Benjamin Ransford, Jacob Sorber und Kevin Fu. „Mementos: System Support for Long-Running Computation on RFID-Scale Devices“. In: Proceedings of the 16th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS XVI). ACM, 2011, S. 159–170. doi: 10.1145/1950365.1950386.
- [28] Brandon Lucia und Benjamin Ransford. „A Simpler, Safer Programming and Execution Model for Intermittent Systems“. In: Proceedings of the 36th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI '15). ACM, 2015, S. 575–585. doi: 10.1145/2737924.2737978.